Cybersecurity Engineer Assessment: Solution Pack

Securing a healthcare AI platform: risk prioritisation, target architecture, controls, incident readiness and executable evidence

Candidate	Talal Rehan
Role assessed	Cybersecurity Engineer, Apex AI Arabia
Prepared	21 September 2026
Deliverable	This PDF plus a supporting evidence folder (detection code, tests and results, Dockerfiles, risk register CSV)

Bottom line. The biggest risks are not exotic AI attacks. They are basic access and exposure failures (no MFA, a shared admin account, a database and admin interface reachable from the wrong places, keys on laptops) plus two AI-specific gaps: patient data in prompts, logs and an external LLM, and agents and RAG that are not constrained by authorization. Eight of the 19 findings score Critical. Five bundled fixes remove the shortest paths to patient data and are achievable within 72 hours. Part 18 backs the detection design with executed evidence: 16 rules, 50 passing tests, all on synthetic data.

How to read. Numbering follows the brief (Parts 1-20, then Bonus). Claims are tagged: **[Executed]** = produced by code I ran (see Part 18); **[Design]** = my recommendation, not yet built; **[Assumption]** = stated in the assumptions note; **[Validate]** = needs qualified legal, compliance or vendor confirmation. Nothing in this pack asserts a fact about Apex that the brief did not provide.

Contents

If I Had One Day to Secure Apex . . . 3
Assumptions and Scope Note . . . 4
Part 1. Risk Assessment and Prioritisation . . . 5
Part 1 (continued). Findings register: all 19 findings . . . 7
Part 2. Target Secure Architecture . . . 9
Part 3. Identity and Access Management Design . . . 12
Part 4. Secrets Incident: A Production API Key Committed Six Weeks Ago . . . 14
Part 5. Should Developers Access the Production Database Directly? . . . 16
Part 6. Container Hardening for AI Services . . . 17
Part 7. API Security Controls for the Four Endpoints . . . 19
Part 8. Vulnerability Management SLA . . . 21
Part 9. Secure CI/CD Pipeline (DevSecOps) . . . 23
Part 10. Logging, Monitoring and Detection Design . . . 25
Part 11. Incident Response: Service Account Reads 25,000 Records in 12 Minutes . . . 28
Part 12. Threat Model for the AI Features . . . 30
Part 13. Secure Retrieval-Augmented Generation (RAG) . . . 32
Part 14. Tool Permissions for the AI Agent . . . 34
Part 15. Data Protection Design . . . 36
Part 16. Mapping to Saudi Requirements and Recognised Frameworks . . . 38
Part 17. Linux Server Hardening: Top 20 Checks . . . 40
Part 18. Hands-On Exercise: Detection Engineering (Executed Evidence) . . . 41
Part 19. Responding to the Executive: "We use HTTPS, a major cloud and a managed database, so we are secure enough." . . . 44
Part 20. Ten Findings the Day Before a Customer Demonstration . . . 46
Bonus. A Respectful Challenge (400 words maximum) . . . 48
AI and Tools Disclosure . . . 49
Appendix A. Live-Defence Preparation: A Surprise Incident . . . 50
Appendix B. Likely Questions in the Live Defence and Short Answers . . . 51
Appendix C. Evidence Index and Files . . . 52

If I Had One Day to Secure Apex

Bottom line first. In one working day, with the security engineer plus the DevOps, backend, AI and IT leads working in parallel, I would close the shortest paths to patient data and to administrative control, stop patient data leaking into logs and an external model, and put the AI agents on a short leash. Every action below is a configuration change or a feature flag, not a rebuild, and each has a **proof**, so "done" means evidenced rather than asserted. The order is deliberate: identity first, because an attacker who controls the cloud console can undo every other fix.

#	Action (what exactly)	Fixes	Owner	Time-box	Proof it worked
1	Enforce MFA on the identity provider, cloud console, Git, CI, VPN and admin UI. Admins first. FIDO2 keys if available; otherwise TOTP today and FIDO2 within two weeks.	#3	IT Lead + Security	2 h admins, 6 h all	IdP report: 100% of privileged users enrolled. A login without MFA is refused (log line).
2	Retire the shared cloud admin account: named admin identities through SSO, no standing admin on daily accounts, shared password and access keys revoked, two sealed break-glass accounts created.	#2	CTO + Security	3 h	Cloud credential report: shared account disabled, zero active keys. Break-glass test login fires an alert.
3	Remove developer IP ranges from the production database firewall. Confirm no public IP; only the app tier can connect.	#7	DevOps Lead	1 h	Before/after rule export. Port test to the DB endpoint from the internet and from a developer IP: closed.
4	Put the admin interface behind ZTNA/VPN plus SSO. Interim: allow-list of VPN egress addresses plus MFA, and block the /admin path at the gateway.	#13	DevOps Lead	2 h	Request to /admin from the public internet is blocked (screenshot and log entry).
5	Rotate every production key held on laptops and the committed key; move to a secrets manager; delete local copies; rotate long-lived CI credentials; turn on push protection for secrets.	#4 #16	DevOps Lead	4 h	Old keys rejected (test call fails). Secrets-manager audit log. History scan shows zero live keys.
6	Stop PHI reaching logs and the external LLM: disable prompt and response body logging in production; restrict the existing log store to two or three named people; add a flag that blocks PHI to the external LLM until the DPA and residency are approved. Do not purge yet: Legal confirms holds first.	#8 #9	Backend + AI Lead + DPO	3 h	Twenty new log lines are metadata only. Egress logs show zero PHI patterns (rule D15).
7	Constrain agents and RAG: disable create_user and delete_record; put update_customer, send_email and issue_refund behind human approval (or off); replace the broad agent credential with a read-only per-tenant token; restrict RAG to a per-tenant index or switch it off for multi-tenant use until authorization-aware retrieval ships.	#14 #15	AI Lead + Security	4 h	Tool-call test matrix: unapproved calls are denied. A cross-tenant canary query returns nothing.
8	Put a gateway and WAF baseline in front of the API: managed rule set, request-size limits, per-key and per-tenant rate limits, per-tenant LLM token budget.	#1 #12	Backend + DevOps	3 h	Load test returns 429 at the limit. WAF logs show blocks on standard probe strings.
9	Centralise minimum viable logs (cloud audit, IdP, gateway, app auth and audit) into one store. Enable five alerts (D02, D03, D06, D07, D13). Write a one-page incident runbook naming on-call, contacts and the notification decision owner.	#11 #18	Security Eng.	5 h	Fire drill: each alert triggered once with a timestamp. Runbook reviewed by the CTO.
10	Start the production database restore test into an isolated environment and record RTO and RPO. Switch on report-only dependency, image and secret scans in CI. Export a first asset inventory from the cloud APIs.	#10 #6 #19	DevOps Lead	1 h to start	Restore job log with elapsed time. First scan report. Inventory export.

Deliberately not in the day: root containers (#5), the vulnerability SLA (#17), full SIEM coverage, and an independent penetration test. None opens a direct path to patient data on its own; they are sequenced in Days 2-30 (Part 1 roadmap). **Risks of moving this fast:** MFA lockouts (mitigated by the sealed break-glass accounts), developers losing database access (mitigated by the just-in-time path in Part 5 and a named DevOps on-call), and a demo or customer feature relying on a tool or index that is switched off (mitigated by feature flags and a rollback note for each change).

Assumptions and Scope Note

The brief describes 19 findings and the AI features of the platform. It does not describe the cloud provider, team size, customer contracts or existing tooling. I have treated anything not stated as **unknown, not as compliant**.

ID	Assumption or limit	Effect on this pack
A1	No facts about Apex beyond the brief were provided. No system was accessed.	All findings are taken as stated; scores are judgement, not measurement.
A2	Cloud provider is not specified.	Design is cloud-neutral. Product names (Trivy, cosign, Semgrep and so on) are examples; equivalents are acceptable.
A3	Apex is a small to mid-size engineering organisation with a CTO, DevOps, backend, AI and IT leads.	Owners are roles, not named people. Effort estimates assume 4-6 people.
A4	The platform is multi-tenant SaaS for healthcare customers in Saudi Arabia and handles personal health data.	Treated as sensitive personal data under PDPL, and as data classified per the customer's scheme. [Validate]
A5	Regulatory mapping is at NCA ECC subdomain and PDPL topic level.	Article and control-level citations, and whether ECC-1:2018 or ECC-2:2024 applies, need confirmation by Compliance/Legal. [Validate]
A6	Likelihood and impact use the scales in Part 1 over a 12-month horizon at the current state.	Percent ranges are guides for consistency, not statistics.
A7	The brief did not supply the ten findings for Part 20.	I defined ten illustrative ones and labelled them. The decision method applies unchanged to real inputs.
A8	Executed evidence exists only for Part 18 (synthetic data). Dockerfiles, YAML and shell commands elsewhere were not run.	Marked [Design]. The Dockerfile was not built because no container runtime was available to me.
A9	One working day is roughly 8-10 hours per person.	Time-boxes in the One-Day plan are estimates to be confirmed by each owner.

Evidence legend used throughout

- **[Executed]** code or tests I ran; outputs are in the evidence folder and quoted in Part 18.
- **[Design]** a recommendation or artefact I wrote but did not deploy.
- **[Validate]** = "Requires legal/compliance validation": depends on legal, regulatory or vendor facts I cannot confirm from here; owner and route are named where it appears.

Part 1. Risk Assessment and Prioritisation

Bottom line. Eight of 19 findings score Critical (20), nine High, two Medium. The Critical set is a coherent attack chain: no MFA plus a shared admin account gives control of the cloud; developer-reachable database and admin interfaces give direct paths to patient data; keys on laptops make credential theft easy; and the AI layer (RAG and agents) can be steered into disclosing or changing data. I fix these first, as five bundles, and schedule the High and Medium items behind them.

1.1 Method

Score = Likelihood x Impact, each 1-5, judged over 12 months at the **current** state (Assumption A6). Priority is not the same as severity: it also reflects how directly the item opens a path to patient data, whether other fixes depend on it, and effort. For example, #9 (external LLM) is High but P0 because it is a live transfer of patient data today.

Score	Likelihood (12 months)	Impact (worst credible outcome)
1	Rare: needs unusual skill or insider; roughly under 5%	Minor: no patient data; under 1 h degradation
2	Unlikely: roughly 5-20%	Moderate: limited internal data or short outage
3	Possible: roughly 20-50%	Serious: partial data of one tenant or over 4 h outage; contract breach
4	Likely: roughly 50-80%; common errors or opportunistic attackers	Severe: patient data of many people or a whole tenant; regulator notification; customer termination risk
5	Almost certain: over 80%, or the condition already exists or is being exploited	Catastrophic: multi-tenant patient data exposure, patient-safety effect, sustained outage, loss of licence or contract

Bands: **Critical** 20-25, **High** 12-19, **Medium** 6-11, **Low** 1-5. Priority: **P0** act within 72 hours, **P1** within 30 days, **P2** within 30-90 days.

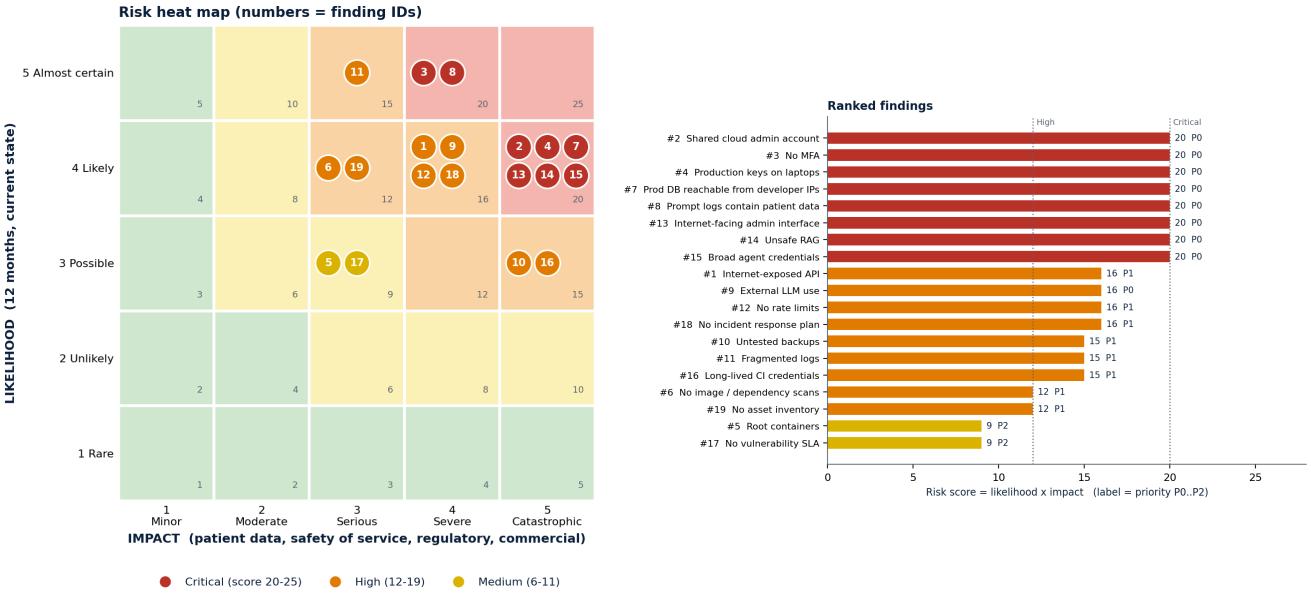


Figure 1.1. Left: heat map. Right: ranked findings with priority. Eight Critical findings all score 20, so ordering inside that band comes from the bundles below.

1.2 The top five fixes (bundled) and why these

Fix	Findings	Why it ranks here	By	Success measure
B1 Identity lockdown	#2, #3	Control-plane compromise nullifies every other fix, so identity goes first.	48 hours	100% of privileged users on MFA; zero shared admin credentials; break-glass alert tested.
B2 Close direct paths to PHI and admin	#7, #13	Minutes of configuration change removes the shortest path to the data store and to admin power.	24-48 hours	External scan shows DB and admin unreachable; firewall diff approved and saved.
B3 Secrets off laptops and CI	#4, #16	Rotate and vault. Removes standing credentials that bypass MFA and network controls.	72 h / 14 d	Old keys rejected; zero static cloud keys in CI; history scan shows no live secrets.
B4 Stop PHI leaking via prompts, logs and external LLM	#8, #9	Live regulatory exposure and the single largest customer-trust risk in a healthcare deal.	72 hours	Rule D15 clean for 7 days; egress logs show no PHI; DPA and residency decision recorded.
B5 Constrain AI agents and RAG	#14, #15	Apex-specific attack surface: injection plus broad power equals data theft or unauthorized actions.	72 h mitigate / 30 d fix	Zero cross-tenant canary hits; unapproved tool calls denied in 100% of tests; agent credentials read-only and scoped.

What I did not put in the top five, and why. Logging (#11), the incident plan (#18) and untested backups (#10) do not reduce the chance of the first compromise; they reduce dwell time and damage. They are important and start on Day 1, but a small, fast set of fixes that removes the shortest paths to patient data comes first. Rate limits (#12) and the WAF (#1) are cheap and included in the one-day list, but they do not by themselves expose patient data. I also chose bundles over single findings because in practice the fixes are done together by the same people (MFA plus shared admin; database plus admin exposure).

1.3 Remediation roadmap

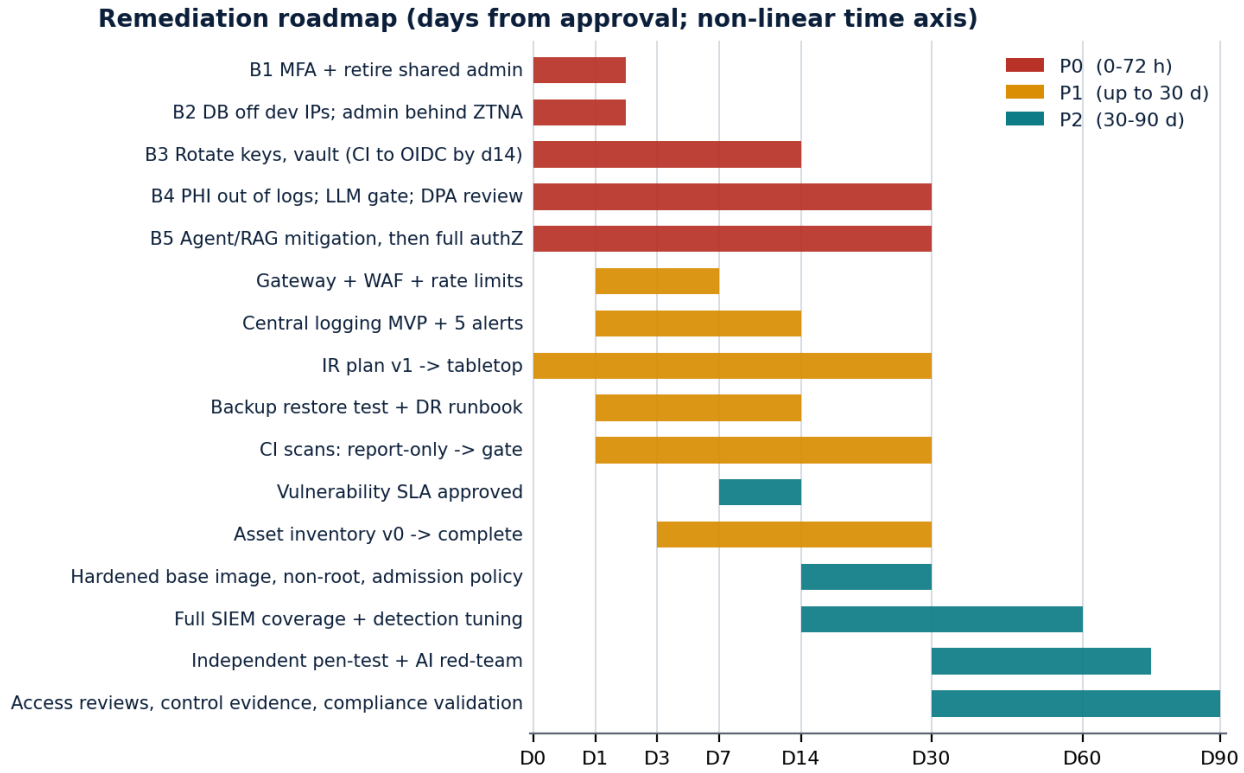


Figure 1.2. 90-day roadmap. P0 bundles run first; P1 items follow; P2 items and independent validation close the loop.

30/60/90 in one line each. Day 30: all P0 and P1 controls live and evidenced, incident tabletop done, CI gates blocking criticals. Day 60: full SIEM coverage with tuned detections, first independent pen test and AI red-team complete, compliance mapping validated by Legal. Day 90: access reviews running, evidence collection automated, metrics reported monthly to the CTO.

Part 1 (continued). Findings register: all 19 findings

#	Finding	Why it matters	L	I	Score	Recommended control	Pri.	Owner	Time
1	**Internet-exposed API**	Public attack surface. Any authorization, injection or abuse flaw is reachable by anyone; no gateway/WAF layer to absorb it.	4	4	16 **High**	API gateway + WAF in front; OIDC authN; object-level authZ; schema validation; origin reachable only from gateway.	**P1**	Backend Lead + Security Eng.	14 days
2	**Shared cloud admin account**	No attribution, no per-person revocation, one leaked credential gives full cloud control. Defeats every other fix.	4	5	20 **Critical**	Named admin identities via SSO; JIT elevation (PAM); sealed break-glass; disable shared account and its keys.	**P0**	CTO + Security Eng.	48 hours
3	**No MFA**	Phishing, credential stuffing or infostealer logs give direct access to cloud, Git, CI and admin UI.	5	4	20 **Critical**	Phishing-resistant MFA (FIDO2/passkey) for admins; MFA for all staff on IdP, cloud, Git, CI, VPN, admin UI.	**P0**	IT Lead + Security Eng.	48 h admins / 7 d all
4	**Production keys on laptops**	Laptop theft, malware or a synced backup exposes production access. Keys are also likely in shell history and dotfiles.	4	5	20 **Critical**	Rotate all; move to secrets manager; workload identity for services; disk encryption + EDR on laptops.	**P0**	DevOps Lead	72 hours
5	**Root containers**	Force multiplier: after any app compromise, root makes container escape and host tampering easier.	3	3	9 **Medium**	Non-root USER, drop capabilities, read-only rootfs, seccomp default; admission policy to enforce.	**P2**	DevOps Lead	30 days
6	**No image / dependency scans**	Python/AI stacks pull large transitive dependency trees; known-vulnerable packages ship unnoticed.	4	3	12 **High**	SCA (pip-audit/OSV), image scan (Trivy/Grype), SBOM in CI; report-only day 1, gate criticals by day 30.	**P1**	DevOps Lead	14 d report / 30 d gate
7	**Prod DB reachable from developer IPs**	Patient data store exposed to the least-controlled endpoints. Stolen laptop or dev credential reaches PHI directly.	4	5	20 **Critical**	Remove IP allow-rules; private subnet only; JIT read-only access through audited DB proxy (Part 5).	**P0**	DevOps Lead	24 hours
8	**Prompt logs contain patient data**	PHI copied into a broadly readable log store: a live PDPL exposure today, independent of any attacker.	5	4	20 **Critical**	Stop body/prompt logging; redact at source; restrict then purge existing logs (after legal-hold check).	**P0**	Backend Lead + DPO	72 hours
9	**External LLM use**	PHI may leave the Kingdom to a third party with unknown retention, training and residency terms.	4	4	16 **High**	Interim gate: no PHI to external LLM. Egress proxy with redaction/tokenization; provider due diligence + DPA.	**P0**	AI Lead + Compliance/DPO	72 h gate / 30 d full
10	**Untested backups**	Unknown recoverability. Ransomware, operator error or corruption becomes an unbounded outage and data-loss event.	3	5	15 **High**	Restore test into isolated env with measured RTO/RPO; immutable, encrypted, separate-account copies; monthly test.	**P1**	DevOps Lead	14 days
11	**Fragmented logs**	Detection blindness and slow investigations; attacker dwell time grows; cannot answer regulators or customers.	5	3	15 **High**	Central pipeline for cloud audit, IdP, gateway, app auth/audit into one SIEM; NTP; retention policy.	**P1**	Security Eng.	14 d MVP / 60 d full
12	**No rate limits**	Scraping, credential stuffing and LLM cost exhaustion (denial-of-wallet) with no brake.	4	4	16 **High**	Gateway rate limits + quotas per user/tenant/key; LLM token budgets; bulk-export caps.	**P1**	Backend Lead	7 days
13	**Internet-facing admin interface**	Highest-privilege application function exposed to brute force, phishing and any auth-bypass bug.	4	5	20 **Critical**	Move behind ZTNA/VPN + SSO + step-up MFA; separate admin host; IP allow-list as interim.	**P0**	DevOps Lead	48 hours
14	**Unsafe RAG**	Retrieval without per-user authorization leaks other users'/tenants' documents; poisoned documents steer the model.	4	5	20 **Critical**	Authorization-aware retrieval (tenant + ACL pre-filter, policy re-check); untrusted-content handling; canary tests.	**P0**	AI Lead + Security Eng.	72 h mitigate / 30 d fix
15	**Broad agent credentials**	A prompt-injected agent inherits broad power: data theft, record changes, account creation.	4	5	20 **Critical**	Per-agent identity; per-tool scoped short-lived on-behalf-of tokens; tool gateway; disable write tools first.	**P0**	AI Lead	72 h mitigate / 30 d fix

#	Finding	Why it matters	L	I	Score	Recommended control	Pri.	Owner	Time
16	**Long-lived CI credentials**	A leaked pipeline secret allows attacker-controlled deployments and supply-chain compromise.	3	5	15 **High**	OIDC federation to short-lived, environment-scoped roles; remove static keys; rotate now.	**P1**	DevOps Lead	14 days
17	**No vulnerability SLA**	Not a breach path itself; causes drift, ad-hoc decisions and unanswerable customer questions.	3	3	9 **Medium**	Approve risk-based SLA and exception process (Part 8); weekly aging report.	**P2**	Security Eng. + CTO	14 days
18	**No incident response plan**	First incident is improvised: slow containment, missed notification deadlines, lost evidence.	4	4	16 **High**	v1 IR plan, on-call, contacts, severity matrix, customer/regulator decision tree; tabletop.	**P1**	Security Eng. + Compliance/DPO	7 d v1 / 30 d tabletop
19	**No asset inventory**	Cannot protect, patch or scope what is not known; shadow assets and orphaned keys persist.	4	3	12 **High**	Auto-generated inventory from cloud APIs, repos and registry; mandatory owner and data-class tags.	**P1**	DevOps Lead	7 d v0 / 30 d complete

L = likelihood, I = impact. Owners are roles (Assumption A3). Editable copy: `risk_register.csv` in the evidence folder.

Part 2. Target Secure Architecture

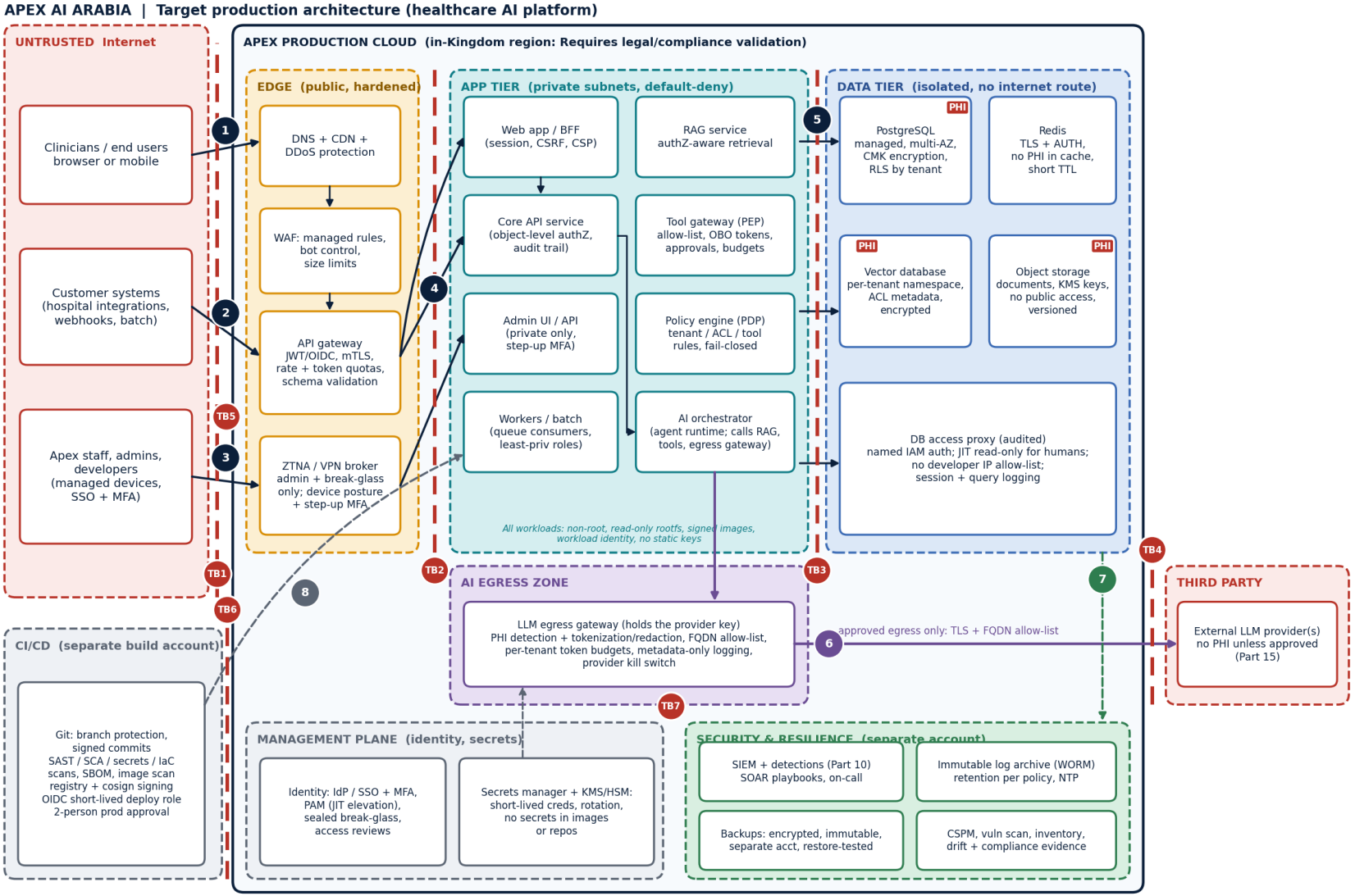


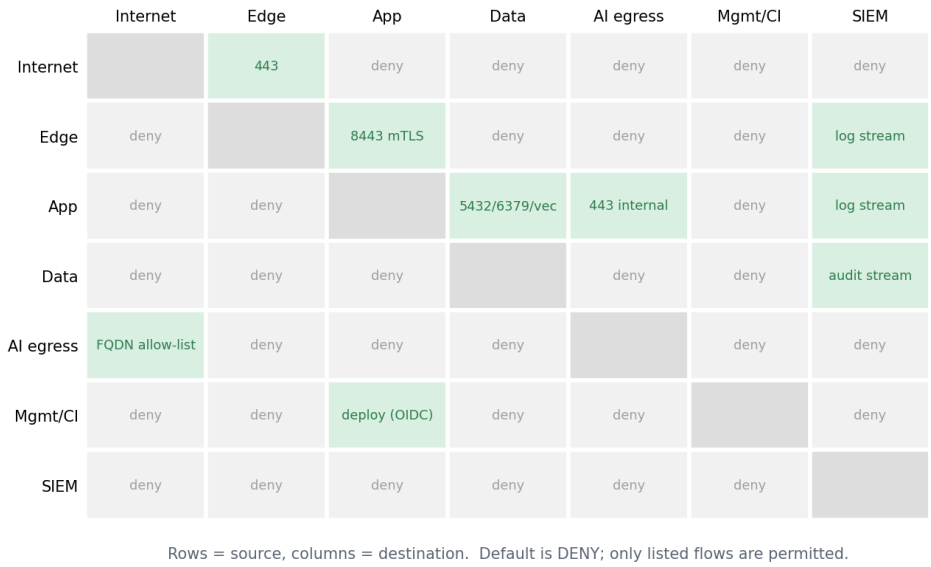
Figure 2.1. Target production architecture with seven trust boundaries (TB1-TB7). PHI tags mark where patient data lives or transits. Numbered flows: 1 users over HTTPS, 2 customer integrations (mTLS or private link), 3 admin through ZTNA, 4 gateway to private app tier, 5 app to data, 6 approved AI egress only, 7 telemetry to SIEM, 8 signed deployments from the build account.

Bottom line. Public traffic reaches only a hardened edge (CDN, WAF, gateway). Everything that touches patient data sits in private tiers with default-deny networking and no human path in except an audited, just-in-time one. AI is treated as its own zone: models are reached only through an egress gateway that redacts, budgets and logs, and agents act only through a tool gateway that enforces policy outside the model. Build, security and backup functions live in separate accounts so that compromise of production does not hand over the evidence or the recovery path.

2.1 Trust boundaries and what protects each

TB	Boundary	Principal threats	Controls at the boundary
TB1	Internet to Edge	Volumetric attack, injection, credential stuffing, scraping	CDN and DDoS protection, WAF managed rules and bot control, TLS 1.2+ (1.3 preferred) with HSTS, request-size limits
TB2	Edge to App tier	Bypass of the edge, forged identity, over-large or malformed requests	Origin reachable only from the gateway; JWT/OIDC validation (issuer, audience, expiry, algorithm allow-list); mTLS to services; schema validation; per-key and per-tenant quotas
TB3	App tier to Data tier	Lateral movement, over-privileged services, data theft by a compromised workload	Per-service workload identity and role; database IAM authentication; TLS with certificate verification; security groups allow only app tier and DB proxy; row-level security by tenant; no internet route
TB4	Platform to third-party AI	PHI leaving the boundary, provider retention, exfiltration through the model channel	Egress gateway with FQDN allow-list; PHI detection and tokenization; per-tenant token budgets; metadata-only logging; provider kill switch; DPA, residency and no-training terms [Validate]
TB5	Staff and admin endpoints to platform	Stolen credentials, unmanaged devices, exposed admin interfaces	ZTNA or VPN broker; SSO with phishing-resistant MFA for admins; device posture check; no public admin endpoints; just-in-time elevation
TB6	CI/CD to Production	Compromised pipeline or dependency, malicious deployment	Separate build account; OIDC short-lived deploy roles; signed artifacts verified at admission; two-person approval; branch protection; pinned actions
TB7	Workloads to management and security planes	Attacker disables logging or reaches secrets and backups	Logs streamed to a separate account (write-only from workloads); immutable archive; backups in a separate account; secrets by short-lived retrieval; alerts on control changes (rules D12, D13)

2.2 Network flow rules



Rows = source, columns = destination. Default is DENY; only listed flows are permitted.

Figure 2.2. Default-deny flow matrix. Only the listed flows exist; everything else is blocked and alerts if attempted.

2.3 Design decisions and their cost

Decision	Why	Trade-off accepted
Separate accounts: production, non-production, security and logs, build, backup	Blast-radius control; attacker in production cannot erase evidence or backups	More setup and cross-account plumbing
Managed database and container platform	Patching and availability offloaded; fewer things to harden	Less low-level control; cost
Egress gateway for all model traffic	One place to redact PHI, enforce budgets, hold the provider key and switch a provider off	Extra hop (tens of milliseconds) and one more component to run
Tool gateway for agents (policy outside the model)	Prompt injection cannot be fully prevented; authorization must not depend on the model behaving	Engineering effort to define tool schemas and policies
Per-tenant vector namespaces	Hard isolation for retrieval; simpler evidence for customers	Higher cost and operational overhead than one shared index
No standing human access to production data	Removes the developer laptop as a path to patient data	Friction in incidents, reduced by break-glass and a fast approval route
Non-production uses synthetic or de-identified data only	Lets developers and demos move fast without PHI risk	Needs a maintained synthetic data set

> **Deliberately left out** to keep this defensible: a service mesh, full micro-segmentation per pod, custom HSM integration, and a multi-region active-active design. They can follow once the fundamentals are evidenced. Hosting location (in-Kingdom region) and any provider registration requirements must be confirmed with Legal and the customer contract. [Validate]

Part 3. Identity and Access Management Design

Bottom line. Every identity type gets its own authentication and authorization model, and **no human holds standing privileged access**. Admins use a separate identity with phishing-resistant MFA and elevate just-in-time; services and pipelines use short-lived workload identity instead of stored keys; AI agents act with the calling user's authority narrowed to one tool at a time. Two sealed break-glass accounts exist for the day the identity system itself fails.

3.1 Identity types

Identity	Authentication	Authorization	Standing access	Lifecycle and review
Workforce	SSO through the IdP; MFA (number-matching push or TOTP minimum); device posture for sensitive apps	Role-based by job family; least privilege; groups sourced from the HR system	Business apps only	Joiner/mover/leaver driven from HR (SCIM); review every 6 months
Privileged admins (cloud, IdP, database)	Separate admin identity, not the daily account; FIDO2 keys mandatory; reached through ZTNA	Just-in-time elevation: ticket, approver, at most 4 h, session recorded	None	Elevation list reviewed monthly; alerts on any grant outside change (D03) and on non-MFA login (D02)
Developers	SSO with MFA from a managed device	Non-production broad; production limited to telemetry without PHI; data only through the JIT path (Part 5)	No production data	Recertified quarterly; access removed the day they leave or change teams
Services and workloads	Cloud workload identity (roles bound to the workload); no static keys; mTLS between services	One service, one role, resource-scoped, deny by default; database IAM authentication	Machine only	Owner reviews quarterly; unused permissions trimmed using access-analysis output; new-network use alerts (D05)
CI/CD	OIDC federation to short-lived tokens (up to 1 h) per environment	Deploy role can update named services only; no IAM administration; production needs approval	None	Pipeline permissions reviewed quarterly; ephemeral isolated runners
AI agents	Own identity per agent; token exchange from the calling user (on-behalf-of); TTL 5-15 min	Tool gateway allow-list with parameter constraints; no ambient cloud credentials (Part 14)	None	New tool needs security review; weekly review of denied and anomalous calls (D09-D11)
Customer users	Federation with the tenant's identity provider where possible; MFA enforced by policy	Tenant-scoped roles plus object-level checks; clinical attributes where needed	Within their tenant	Tenant administrators review their users; Apex supplies review reports
Contractors and vendors	Time-boxed accounts with a named sponsor; MFA	Least privilege; no production by default	No	Auto-expire in 90 days or less; sponsor recertifies

3.2 Privileged access: just-in-time and break-glass

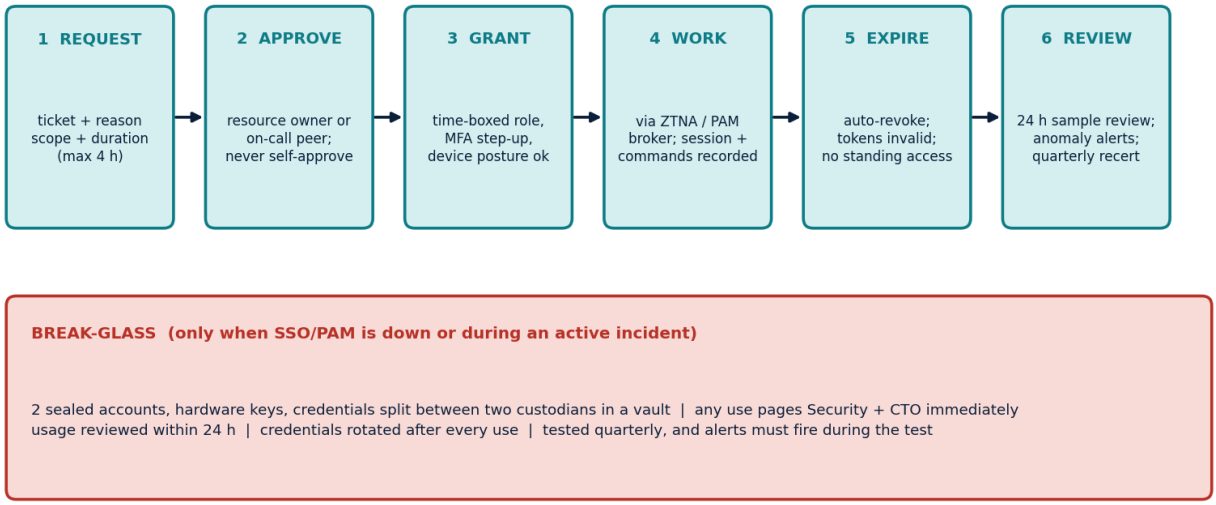


Figure 3.1. JIT privileged access lifecycle, with the break-glass exception underneath.

3.3 Joiner, mover, leaver and access reviews

Event	Trigger	Action	Target time
Joiner	HR record created	Account created by SCIM with baseline role for the job family; extra access needs manager plus resource-owner approval; MFA enrolled before first sign-in	Day 1
Mover	HR role or team change	Old-role groups removed automatically; new access requested; any privileged access re-approved; ownership of repos and services reassigned	Removal same day
Leaver	HR termination	IdP account disabled and sessions and tokens revoked; devices recovered; secrets the person knew are rotated; break-glass custody changed if applicable	Within 1 h of effective time; immediate if involuntary
Contractor end	Sponsor date or 90 days	Automatic expiry unless sponsor recertifies	On expiry
Access review	Calendar	Privileged monthly; production and PHI-adjacent quarterly; everything else every 6 months. Unreviewed access is removed after a 14-day grace period	Evidence kept for audit

Rules I would enforce without exception: no shared accounts; no admin work from the daily identity; no self-approval of privileged access; no static cloud keys for workloads; MFA methods that resist push fatigue (number matching, conditional access, FIDO2 for admins); and break-glass use always pages a human and is reviewed within 24 hours.

Part 4. Secrets Incident: A Production API Key Committed Six Weeks Ago

Bottom line. Treat the key as compromised for the whole six weeks. Contain first by replacing and revoking it, then find out whether it was used, then clean up every place it could still live, and only then fix the process. Deleting the commit is **not** containment: the key survives in git history, clones, forks, CI logs, build caches and any image that copied the source.

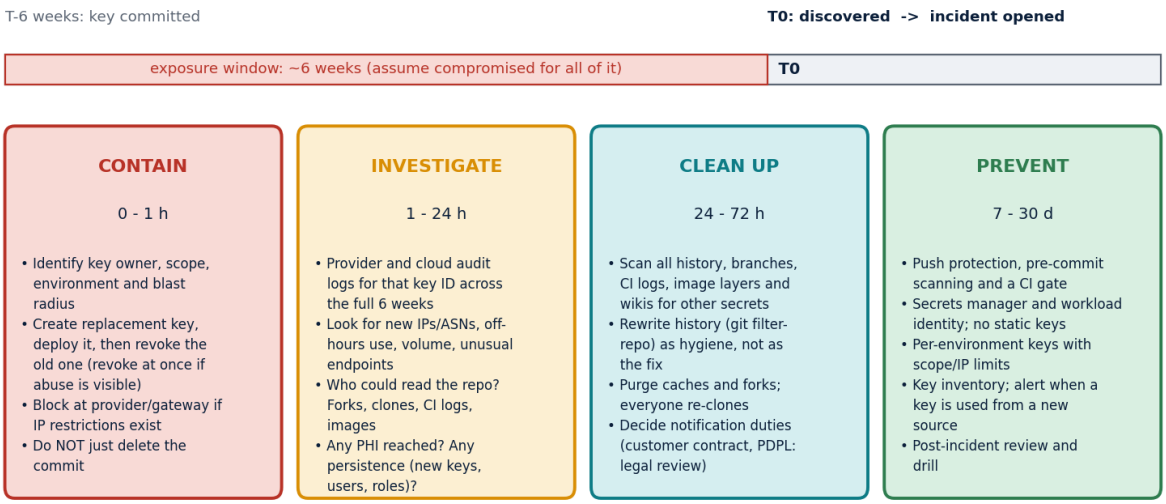


Figure 4.1. Response phases for the committed key.

4.1 First hour: revoke or rotate?

Decision rule: if there is **any sign of misuse**, or the repository was ever public, revoke immediately and accept a short outage. If there is no sign of misuse and revoking would interrupt patient-facing service, create the replacement, deploy it, and revoke the old key within an hour. Never wait for the investigation before rotating.

4.2 What I would investigate

Question	Evidence to check	If yes
Was the key used by anyone else?	Provider or API usage logs and cloud audit logs filtered by key ID for all six weeks: new IPs and ASNs, geographies, user agents, off-hours use, request volume, unusual endpoints	Open a security incident (Part 11); scope data reached
What could the key do?	Key scope, environment, rate limit, IP restrictions, resources reachable (data, other secrets)	Rotate everything it could read; assess PHI exposure
Did the attacker persist?	New keys, users, roles, webhooks, scheduled jobs, changed IAM policies since the commit date	Remove; widen the investigation
Who could see the repository?	Repository visibility history, collaborators, forks, mirrors, deploy tokens, CI runners, package artifacts	Treat all of them as exposed; purge forks and caches
Are there other secrets?	Scan full history on all branches, CI logs and artifacts, container image layers, wikis and tickets	Rotate each; add to the incident record
Are we obliged to notify?	Whether personal data was accessed; customer contract terms; PDPL breach duties	Legal and the DPO decide; the clock starts when Apex became aware [Validate]

4.3 Commands (illustrative, [Design])

```
# locate every occurrence, all branches and tags
git log --all -S'<KEY_PREFIX>' --oneline
gitleaks detect --source . --log-opts="--all" --report-format json --report-path leaks.json
# after rotation only: rewrite history for hygiene (does not restore secrecy)
git filter-repo --replace-text expressions.txt
git push --force --all && git push --force --tags
# then: everyone re-clones; purge CI caches; delete forks; check built images
docker history --no-trunc <image> | grep -i '<KEY_PREFIX>'
```

4.4 Prevention

- **Stop it at the keyboard:** pre-commit secret scanning and server-side push protection that rejects the push.
- **Stop it in the pipeline:** secret-scan gate in CI (Part 9); block on any finding.
- **Remove the need:** secrets manager and workload identity so applications fetch short-lived credentials; no keys in environment files or images.

- **Shrink the value:** per-environment keys with scope and IP limits; automatic rotation; alerts when a key is used from a new source.
- **Build the habit:** secrets inventory with owners; a 30-minute runbook drill twice a year; blame-free reporting so people report quickly.

What I would not do: announce the key in a public or broad chat; rotate without checking for dependent services; treat history rewriting as the fix; or close the incident before the provider logs for the full six weeks have been reviewed.

Part 5. Should Developers Access the Production Database Directly?

Bottom line: no, not by default and never from a laptop over an IP allow-list. Developers do their normal work against non-production data. For the rare case that needs production data, there is one audited path: request, approve, time-boxed read-only access through a broker, session recorded. Schema changes go through the pipeline. This keeps engineers effective and removes the developer laptop as a route to patient data.

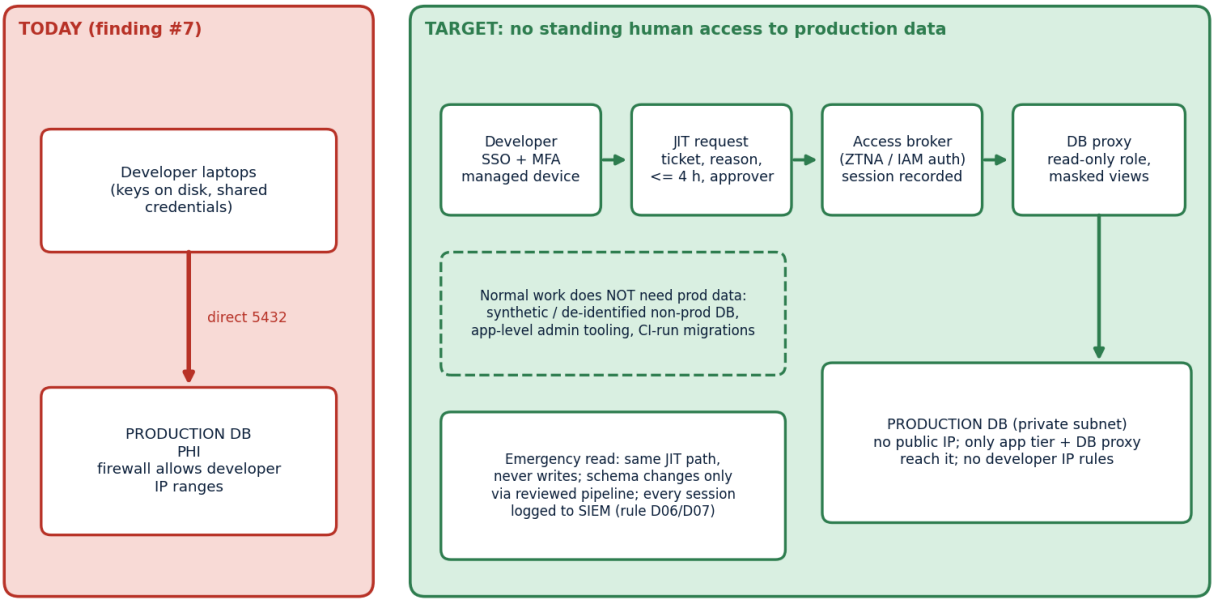


Figure 5.1. Today (finding #7) versus the target access model.

5.1 What replaces direct access

Need	Path	Control
Build and debug features	Non-production environment with synthetic or de-identified data	No PHI outside production; data generator maintained
Look at a customer issue	Application-level admin tooling with masked fields and per-record audit	Object-level authorization; reason required; alerts on volume (D06)
Run a migration	Reviewed change through the pipeline using a migration role	Two-person approval; role exists only for the run; DDL never granted to the app role
Emergency data fix or forensic read	Just-in-time request to the DB proxy: read-only, up to 4 h, named IAM authentication	Approver is not the requester; session and queries logged; masked views by default; review within 24 h
Database on-call (DBA)	Same JIT path; elevated only for the incident	Same logging; break-glass only if the broker is down

5.2 Database hardening that makes the model hold

- Private subnet, no public address; security groups allow only the app tier and the DB proxy (no developer IP rules, ever).
- IAM-based authentication with short-lived tokens for humans; no shared database passwords; app credentials fetched from the secrets manager and rotated automatically.
- TLS required with certificate verification; encryption at rest with a customer-managed key; encrypted backups in a separate account.
- Least-privilege roles: application role has no DDL and no superuser; separate migration role; row-level security by tenant; statement and idle timeouts.
- Audit logging (for example pgaudit) shipped to the SIEM; alerts on bulk reads and on any direct connection from a non-approved source.

Rollout without breaking people. The One-Day plan removes developer IP rules immediately. To avoid a stall, a named DevOps on-call runs JIT requests that day, and within a week the read-only masked-views path and synthetic data set replace the common reasons developers used direct access. Fourteen days of connection logs (taken before the rule change if available) tell us which use cases to serve first.

Part 6. Container Hardening for AI Services

Bottom line. A container is a process boundary, not a security boundary, so the goal is to make a compromised container boring: non-root, read-only, no capabilities, no secrets, no route to anything it does not need. AI services add specific risks: unsafe model loading, code-execution tools, and provider keys.

6.1 Checklist (18 items)

Area	Check	Why
Build	Minimal base image (slim or distroless), pinned by digest, rebuilt weekly	Smaller attack surface; reproducible; picks up patches
Build	Multi-stage build; no compilers, package managers or shells in the final image where practical	Less for an attacker to use
Build	Dependencies from a hashed lock file (`--require-hashes`)	Blocks dependency substitution
Build	No secrets in ENV, ARG, layers or the build context; `.dockerignore` excludes .git, .env, keys	Layers are readable by anyone with the image
Build	SBOM generated, image scanned, image signed; only signed digests admitted to production	Supply-chain control (Part 9)
Runtime	Non-root numeric user; `allowPrivilegeEscalation: false`	Limits impact of an application compromise
Runtime	Read-only root filesystem with small memory-backed tmp volume	Stops persistence and tampering
Runtime	Drop all Linux capabilities; default seccomp profile; AppArmor or SELinux enforcing	Shrinks the kernel attack surface
Runtime	No privileged mode, hostPath, host network or docker.sock mounts	Each is a direct route to the host
Runtime	CPU, memory and PID limits	Stops runaway or abusive workloads
Runtime	Default-deny network policy; egress only to what the service needs	Contains exfiltration and lateral movement
Runtime	Service account token not auto-mounted unless required	Removes a common pivot
Runtime	Admission policy enforces all of the above (for example Kyverno or OPA Gatekeeper)	Turns guidance into a control
AI-specific	Load models only from a trusted registry with checksum verification; prefer safe formats (safetensors) over pickle-based ones	Unsafe deserialization can run attacker code on load
AI-specific	Code-execution or browsing tools run in a separate sandbox (gVisor or microVM), no network, no credentials	Agent tools are attacker-influenced code paths
AI-specific	LLM provider keys exist only in the egress gateway, fetched from the secrets manager	One place to protect and rotate
AI-specific	Model cache and vector store credentials are per-tenant scoped and read-only where possible	Limits cross-tenant reach
Operations	Runtime detection (for example Falco or EDR agent) and log shipping; weekly rebuild and redeploy	Detects abuse; keeps patches fresh

6.2 An insecure Dockerfile and what is wrong with it

```
FROM python:latest
WORKDIR /app
COPY . .
ENV OPENAI_API_KEY=sk-live-EXAMPLE-DO-NOT-USE
ENV DATABASE_URL=postgres://admin:Password123@prod-db:5432/app
RUN apt-get update && apt-get install -y curl wget vim openssh-server netcat
RUN pip install -r requirements.txt
RUN curl -sSL https://example.com/install.sh | bash
EXPOSE 22 8000 5432
USER root
RUN chmod -R 777 /app
CMD ["python", "app.py"]
```

Line	Problem	Fix
`FROM python:latest`	Unpinned, large, changes under you; unknown vulnerabilities	Slim base pinned by digest; weekly rebuild
`COPY . .`	Copies .git, .env, keys and tests into the image	.dockerignore; copy only src/
`ENV OPENAI_API_KEY / DATABASE_URL`	Secrets baked into image metadata; readable by anyone who can pull it	Fetch at runtime from the secrets manager; use workload identity
`apt-get install curl wget vim openssh-server netcat`	Attacker toolkit and an SSH daemon in the image	Install nothing; use the platform for shell access if ever needed
`RUN curl ... | bash`	Executes unverified remote code at build time	Vendor and checksum-verify, or remove
`EXPOSE 22 8000 5432`	Advertises SSH and a database port on an app container	Expose only 8000
`USER root`	Any compromise is root in the container	Numeric non-root user
`chmod -R 777 /app`	World-writable code and data; easy tampering	Owned by the app user; read-only root filesystem

Line	Problem	Fix
`CMD python app.py`	No health check; development server assumptions	Production server, health check, explicit entrypoint

6.3 The corrected version

```
# syntax=docker/dockerfile:1.7
# Pin by digest in real use: python:3.12-slim-bookworm@sha256:<DIGEST_FROM_REGISTRY>
FROM python:3.12-slim-bookworm AS builder
ENV PIP_NO_CACHE_DIR=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /build
COPY requirements.lock .
# requirements.lock is generated with hashes (pip-compile --generate-hashes)
RUN python -m venv /opt/venv && /opt/venv/bin/pip install --require-hashes -r requirements.lock

FROM python:3.12-slim-bookworm
RUN groupadd --gid 10001 app && \
    useradd --uid 10001 --gid 10001 --no-create-home --shell /usr/sbin/nologin app
COPY --from=builder /opt/venv /opt/venv
WORKDIR /app
COPY --chown=10001:10001 src/ ./src/
ENV PATH="/opt/venv/bin:$PATH" PYTHONUNBUFFERED=1
USER 10001:10001
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD ["python","-c","import urllib.request;urllib.request.urlopen('http://127.0.0.1:8000/healthz',timeout=
ENTRYPOINT ["python","-m","uvicorn","src.main:app","--host","0.0.0.0","--port","8000"]
```

6.4 Runtime constraints and how to verify

```
apiVersion: v1
kind: Pod
metadata:
  name: ai-api
spec:
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: api
      image: registry.example.com/apex/ai-api@sha256:<DIGEST>
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
      resources:
        limits: { cpu: "1", memory: 1Gi }
      volumeMounts:
        - { name: tmp, mountPath: /tmp }
  volumes:
    - { name: tmp, emptyDir: { medium: Memory, sizeLimit: 64Mi } }

# verification commands (run in CI and after deploy)
hadolint Dockerfile                                  # lint
trivy image --severity HIGH,CRITICAL --exit-code 1 <image>   # vulnerability gate
docker run --rm <image> id                           # expect uid=10001, not root
docker inspect <image> --format '{{.Config.Env}}'    # expect no secrets
kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>  # expect minimal
```

Honest status: [Design]. I did not build this image because no container runtime was available to me. The Dockerfile, `.dockerignore` and pod spec are in the evidence folder. Before use: generate the hashed lock file, pin the base digest, and let `hadolint` and `trivy` run in CI.

Part 7. API Security Controls for the Four Endpoints

Bottom line. Most API breaches in multi-tenant systems are broken authorization, not clever exploits. The controls therefore lead with **object-level and function-level authorization**, then abuse resistance (rate limits, size caps, bulk-export limits), then input and output handling, then audit. The 25,000-records-in-12-minutes scenario in Part 11 is exactly what controls C4 and C11 are designed to make hard.

7.1 Control set (16 controls)

ID	Control	Threat addressed	How I would verify
C1	Authenticate every request with short-lived OIDC tokens; validate issuer, audience, expiry and algorithm allow-list; reject alg=none; cache JWKS safely	Spoofed or stolen tokens	Token-tampering test suite; expired and wrong-audience tokens rejected
C2	**Object-level authorization** on every read and write: tenant and entitlement checked server-side on the resource ID; return 404 for unauthorised objects; IDs are unguessable but never the control	BOLA / IDOR (OWASP API1)	Automated test: user A requests B's object across every endpoint; expect 404
C3	**Function-level authorization**: role plus scope checked per operation; admin functions on a separate private host	Privilege escalation (API5)	Role-matrix tests; admin host unreachable from the internet
C4	Rate limits and quotas per user, tenant and key; separate limits for token spend, tool steps and bulk reads	Scraping, brute force, denial-of-wallet (API4)	Load test returns 429 at the limit; budget alerts (D16)
C5	Strict schema validation; reject unknown fields; max body size and field length; content-type allow-list	Injection, mass assignment (API3, API6)	Schema fuzzing (for example Schemathesis) in CI
C6	Property-level authorization: callers cannot set role, tenant or approval fields	Mass assignment / BOPLA	Negative tests that submit privileged fields
C7	Output handling: response schemas, field masking by role, no stack traces; treat model output as untrusted (encode, sanitise, no auto-loaded remote content)	Data exposure; XSS; LLM05	Response-schema tests; CSP violation reports
C8	Prompt-injection design at the assistant endpoint: system and user roles separated, retrieved content marked untrusted, policy enforced outside the model	LLM01, LLM06	Injection regression set run in staging (Part 9)
C9	Agent execution only through the tool gateway: allow-list, parameter constraints, on-behalf-of token, approvals, step and time budgets, idempotency key	Excessive agency, unsafe actions	Tool-call matrix tests; denied-call logging
C10	Audit logging: who, what object, decision, purpose, source; identifiers only, never PHI content; write-once shipping	Non-repudiation; forensics	Log-completeness test; PHI pattern scan (D15)
C11	Bulk-access limits: page size caps, export needs approval, per-principal record budgets, anomaly alert	Mass data extraction	Replay of the 25,000-record scenario returns 429 or alert
C12	Transport and headers: TLS 1.2+ (1.3 preferred), HSTS, strict CORS allow-list, CSRF protection for cookie sessions, secure headers	Interception, cross-site abuse	Scanner and header tests
C13	File and document handling: type and size checks, malware scan, no execution, short-lived signed URLs, content-disposition set	Malicious uploads; poisoned RAG	Upload abuse tests; quarantine path tested
C14	Signed webhooks and replay protection for customer integrations (HMAC plus timestamp), mTLS or private link where possible	Forgery and replay	Replay test with old timestamp is rejected
C15	WAF managed rules and bot control in front of the gateway; virtual patching path	Commodity attacks	Probe strings blocked; monitored false-positive rate
C16	API inventory and contract: OpenAPI as source of truth; no undocumented endpoints; deprecation policy	Shadow and zombie APIs (API9)	Route-versus-spec diff in CI

7.2 Endpoint-by-control matrix

Control	POST /assistant/message	GET,PUT /documents/{id}	POST /admin/users	POST /agent/execute
C1	✓	✓	✓	✓
C2		✓		
C3			✓	✓
C4	✓	✓	✓	✓
C5	✓	✓	✓	✓
C6			✓	✓
C7	✓	✓		
C8	✓			✓
C9				✓
C10	✓	✓	✓	✓
C11	✓	✓		
C12	✓	✓	✓	✓
C13		✓		
C14				✓
C15	✓	✓	✓	✓
C16	✓	✓	✓	✓

7.3 Endpoint specifics

POST /assistant/message

- Tenant and user come from the verified token, never from the request body or the prompt.
- Per-tenant token budget, maximum message size, and maximum conversation length; streaming responses are also metered.
- Retrieval is authorization-aware (Part 13); output is filtered for PHI and for citations that point to unauthorised documents.

GET / PUT /documents/{id}

- Check tenant and entitlement on every call, including PUT; a PUT must not be able to change the owner, tenant or ACL fields (C6).
- Uploads are scanned and quarantined until classified; downloads use short-lived signed URLs; deletion propagates to embeddings.

POST /admin/users

- Served only from the private admin host behind ZTNA; step-up MFA required (check the authentication-context claim).
- A caller cannot grant a role higher than their own; privileged roles need a second approver; every creation emails the owner and raises an audit event.
- Burst creation alert (D04) and out-of-change privilege grants alert (D03).

POST /agent/execute

- Runs through the tool gateway only: allow-listed tool, validated parameters, on-behalf-of token, maximum steps and time.
- Human-approved and prohibited tool classes are enforced server-side (Part 14). The approval token is bound to the exact payload hash and expires in minutes.

Illustrative object-level check ([Design])

```
@router.get("/documents/{doc_id}")
def get_document(doc_id: UUID, ctx: AuthContext = Depends(authenticate)):
    doc = repo.get(doc_id)                          # fetch without trusting the caller
    if doc is None or doc.tenant_id != ctx.tenant_id or not policy.can(ctx, "read", doc):
        audit.deny(ctx, "documents.read", doc_id)   # log the decision, not the content
        raise HTTPException(status_code=404)         # same answer for missing and forbidden
    audit.allow(ctx, "documents.read", doc_id)
    return DocumentOut.model_validate(doc)          # response schema strips internal fields
```

Part 8. Vulnerability Management SLA

Bottom line. Use a risk-adjusted score, not CVSS alone, to place each finding in an SLA tier. CVSS measures how bad a flaw could be in general; it does not know whether the flaw is being exploited, whether the vulnerable code is reachable, whether it is exposed to the internet, or whether patient data is behind it.

8.1 SLA tiers

Tier	Typical criteria (risk-adjusted score)	Mitigate by	Fix by	Escalation
Emergency	9.0-10: known exploited or trivially exploitable, on an internet-facing or PHI-adjacent asset	24 hours	72 hours	CTO informed at once; incident channel opened
Critical	7.0-8.9	72 hours	7 days	Weekly to CTO
High	5.0-6.9	n/a	30 days	Weekly report
Medium	3.0-4.9	n/a	90 days	Monthly report
Low	Below 3.0	n/a	180 days or next planned release	Backlog review

The clock starts at the earlier of vendor publication and internal detection. **Exceptions** need the risk owner's (CTO) written approval, a compensating control, and an expiry no longer than 90 days; expired exceptions reopen automatically. Where no context exists yet, default mapping: CVSS 9.0+ to Critical, 7.0-8.9 to High, and so on, then adjust upward for exploitation or exposure.

8.2 Why CVSS alone is not enough

- **Exploitation evidence:** a medium flaw exploited in the wild outranks a critical one nobody can reach (use known-exploited lists and exploit-probability scores as inputs).
- **Exposure:** internet-facing, internal, or isolated build-only assets carry different real risk.
- **Data at stake:** a flaw next to patient data matters more than one next to public content.
- **Reachability:** a vulnerable library function that the code never calls is much less urgent than one on a live request path.
- **Compensating controls:** a WAF rule, network isolation or MFA changes the urgency, though it does not remove the flaw.
- **Chaining:** two mediums can form a critical path; base scores do not see that.
- **AI-specific weaknesses** (prompt injection, unsafe agent tools, cross-tenant retrieval) often have no CVE at all and must still enter the same queue.

8.3 The Apex prioritisation method ([Design], weights to be tuned)

Risk-adjusted score = CVSS x E x X x D x R x C, capped at 10. Factors:

Factor	Values
E exploitation evidence	known exploited = 1.4, public PoC / trivial = 1.15, none known = 0.9
X exposure	internet-facing = 1.25, internal = 1.0, isolated / build-only = 0.7
D data at stake	PHI / crown jewels = 1.25, internal data / privileged = 1.0, no sensitive data = 0.8
R reachability	reachable = 1.0, not reachable = 0.4
C compensating control	none = 1.0, compensating control = 0.7

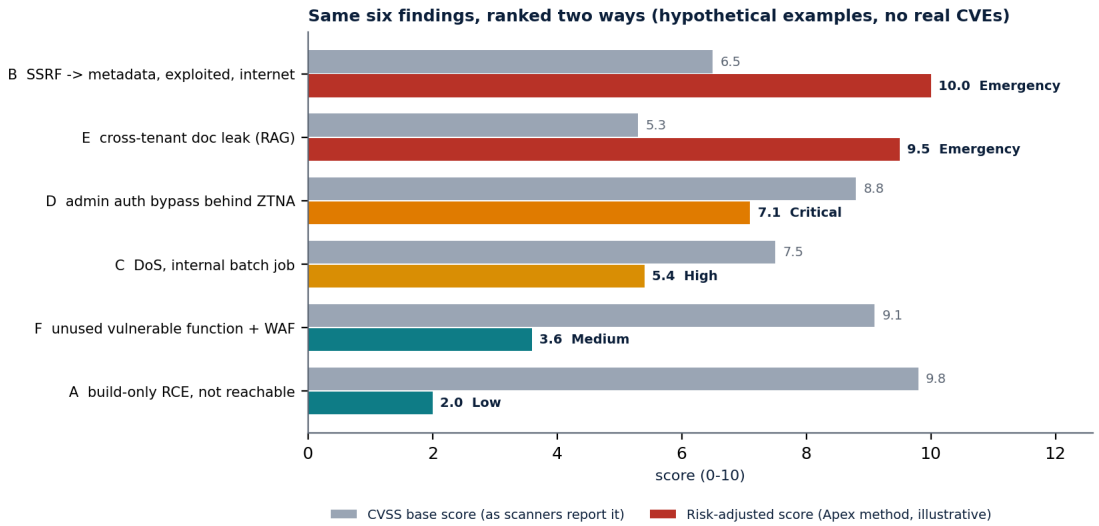


Figure 8.1. Six hypothetical findings ranked by CVSS and by the risk-adjusted score. No real CVE identifiers are used.

ID	Hypothetical finding	CVSS	Adjusted	Tier	Reasoning
A	Critical RCE in a dev-only build dependency	9.8	2.0	Low	Not reachable, build-only: low urgency despite a critical CVSS score.
B	SSRF on internet-facing API reaching cloud metadata (exploited in the wild)	6.5	10.0	Emergency	Exploited, internet-facing, PHI reachable: emergency even though CVSS is only medium.
C	DoS in library used by internal batch job	7.5	5.4	High	Availability only, internal batch job: patch in normal cycle.
D	Auth bypass in internal admin tool (behind ZTNA + MFA)	8.8	7.1	Critical	Serious but behind ZTNA and MFA: critical tier, 7-day fix.
E	Info leak exposing other tenants' documents via RAG endpoint	5.3	9.5	Emergency	Trivial to exploit and exposes other tenants' PHI: emergency.
F	Critical flaw in library present in image, vulnerable function never called (WAF rule added)	9.1	3.6	Medium	Function never called and WAF rule in place: medium, fix at next release.

8.4 Operating rhythm and measures

- Dependency and secret scans on every pull request and daily on main; image rebuild and scan weekly; external attack-surface scan weekly; independent penetration test yearly and after major change.
- Measures: percent closed within SLA by tier, mean time to remediate, open items by age, open exceptions and their expiry, scanner coverage of the asset inventory.
- Ownership: every finding has a named service owner; Security tracks and escalates but does not own the fix.

Part 9. Secure CI/CD Pipeline (DevSecOps)

Bottom line. Security checks run where developers already work, fail fast on the few things that must never ship, and produce a signed artifact that production verifies before it runs. The pipeline itself is treated as a production system, because a stolen pipeline credential is a stolen production.

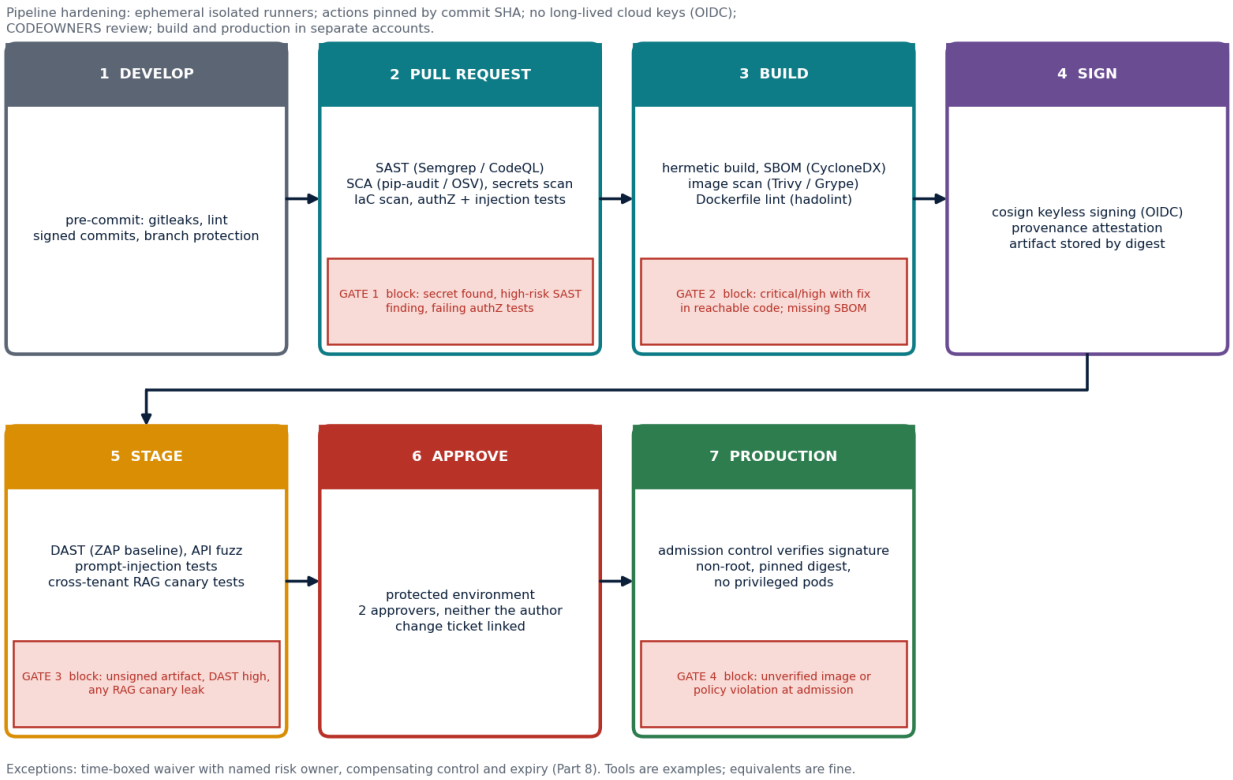


Figure 9.1. Pipeline stages with four blocking gates.

9.1 Gates and thresholds

Gate	Check	Blocks on	Exception path
1 Pull request	Secret scan; SAST; IaC scan; authorization and injection unit tests	Any live secret; high-severity SAST finding; failing authZ test	Time-boxed waiver by Security Engineer with expiry
2 Build	Dependency scan (reachability-aware where possible); image scan; SBOM present; Dockerfile lint	Critical or high with a fix available in reachable code; missing SBOM; root user	Waiver with compensating control (Part 8)
3 Stage to production	Signature exists; DAST baseline; API contract fuzz; prompt-injection regression; cross-tenant RAG canary tests	Unsigned artifact; DAST high; any canary leak	None for canary leaks
4 Admission	Cluster verifies signature and digest; policy: non-root, no privileged, resource limits	Unverified image; policy violation	Break-glass deploy with page and next-day review

9.2 Protecting the pipeline itself

- Branch protection, CODEOWNERS review, signed commits; two approvers for production and neither is the author.
- Ephemeral, isolated build runners; no long-lived cloud keys (OIDC federation to short-lived, environment-scoped roles).
- Actions and plugins pinned by commit SHA; dependencies through an internal proxy or mirror; build and production in separate accounts.
- Secrets used by the pipeline are scoped per environment and never printed; pipeline logs are scanned for secrets.
- Developer experience: PR checks under about 10 minutes, one-click triage of false positives within two working days, and paved-road templates so teams inherit the controls.

9.3 Illustrative workflow ([Design], not executed)

```
name: ci
on: { pull_request: {}, push: { branches: [main] } }
permissions: { contents: read }          # least privilege by default
jobs:
  security:
    runs-on: ubuntu-latest
    permissions: { contents: read, id-token: write, security-events: write }
    steps:
      - uses: actions/checkout@<COMMIT_SHA>         # pin actions by commit SHA
        with: { fetch-depth: 0 }
      - run: gitleaks detect --redact --exit-code 1  # GATE 1: secrets
      - run: pip-audit -r requirements.lock --strict # GATE 2: known-vulnerable deps
      - run: semgrep ci --config p/owasp-top-ten     # SAST
      - run: docker build -t app:${{ github.sha }} .
      - run: trivy image --severity HIGH,CRITICAL --exit-code 1 app:${{ github.sha }}
      - run: syft app:${{ github.sha }} -o cyclonedx-json > sbom.json
      - run: cosign sign --yes <registry>/app@<digest>   # keyless, OIDC identity
```

Part 10. Logging, Monitoring and Detection Design

Bottom line. Today logs are fragmented, so the first job is one pipeline for the fifteen sources below, with a heartbeat on each so silence is itself an alert. On top of that sit 16 detections that cover suspicious access, privilege change, credential use, data exfiltration, anomalous agent behaviour and control failure. All 16 are implemented and tested in Part 18. Logs record **who, what, which object, decision and volume**, never patient content.

10.1 Log sources

#	Source	Why it matters	Key fields	Rules	Retention
1	Cloud control-plane audit	Who changed identity, network, keys, logging	actor, action, resource, source IP, MFA	D03 D04 D13	12 mo
2	IdP / SSO	Logins, MFA, conditional access, admin changes	user, result, MFA method, IP, device	D01 D02	12 mo
3	API gateway and WAF	Front-door traffic and blocks	route, status, tenant, user, bytes, rule ID	D06 D12	12 mo
4	Application audit	Business-level access decisions	user, tenant, object ID, action, decision, records	D06 D07	12 mo
5	Database audit and DB proxy	Direct data access and JIT sessions	role, statement class, tables, rows, duration	D06	12 mo
6	Tool gateway (agents)	Every agent action and approval	agent, tool, parameter hash, approval ID, on-behalf-of user	D09 D10 D11	12 mo
7	LLM egress gateway	What leaves for the model, and how much	tenant, model, token counts, redaction counts, destination	D08 D16	12 mo
8	RAG retrieval audit	What was retrieved for whom	user, query hash, doc IDs returned or denied, decision	D11	12 mo
9	Kubernetes / container runtime	Exec, admission denials, runtime alerts	API audit, policy denials, Falco events	D13	12 mo
10	Network flow, DNS, egress proxy	Lateral movement and exfiltration	src, dst, port, bytes; DNS queries	D07 D08	90 d hot, then archive
11	CI/CD and Git audit	Code, pipeline and deploy integrity	pushes, approvals, runs, scanner results, deploys	D14	12 mo
12	Secrets manager and KMS	Secret reads and decrypt events	principal, secret or key ID, source	D05	12 mo
13	Endpoint / EDR	Laptop and server compromise	process, detection, disk-encryption state	n/a	90 d
14	Backup and restore jobs	Recoverability evidence	status, size, restore-test result	n/a	12 mo
15	Log-pipeline heartbeat	Detects silence and tampering	per-source event counts each minute	D12	30 d

Retention of 12 months searchable is a proposal to be validated against the applicable NCA ECC logging requirement and customer contracts. [Validate] Logs are written to a separate account with write-only permission from workloads, and clocks are synchronised.

10.2 Detection catalogue (16 rules)

ID	Detection	Logic (starting threshold)	Source	Sev.	First response	Mapping
D01	**Brute force followed by successful login** Suspicious access	10 or more failed logins from one IP within 5 min, then a success within 10 min	IdP / auth	High	Block source IP; force password reset and session revoke	T1110
D02	**Privileged login without MFA** Suspicious access	Successful login by a privileged role with mfa=false	IdP / auth	High	Terminate session; require MFA re-enrolment; page Security	T1078
D03	**Privileged role granted outside approved change** Privilege change	Grant of a privileged role with no change ticket, or approver equals actor	Cloud audit	High	Auto-revert grant; page Security; review approver	T1098
D04	**Burst account creation** Privilege change	5 or more account creations by one actor within 10 min	Cloud audit	High	Suspend the new accounts; review creator	T1136
D05	**Service account used from previously unseen network** Credential use	Service account seen from a /24 not in its 12-hour baseline	Auth + API	High	Revoke tokens; rotate the service secret	T1078.004
D06	**Mass record access** Data exfiltration	More than 300 records per 10 min (human) or 5,000 (service; per-actor overrides)	App audit / API	High	Auto-throttle then revoke credential; page Security	T1213
D07	**Bulk read followed by egress to unapproved destination** Data exfiltration	D06 alert then connection to a non-allow-listed destination within 30 min	App audit + egress	Critical	Block destination; isolate workload; open SEV-1	T1041
D08	**Egress to unapproved destination** Data exfiltration	Any egress from a workload to a destination outside the allow-list	Egress / flow logs	Medium / High	Block destination; investigate workload	T1048
D09	**Agent called prohibited / unapproved high-risk tool** Anomalous agent action	Prohibited tool called, or approval-required tool called without approval token	Tool gateway	High/ Critical	Block call; disable tool by flag; review session	OWASP-LLM06
D10	**Agent anomalous volume / customer enumeration** Anomalous agent action	More than 30 tool calls or 20 distinct customers per agent in 5 min	Tool gateway	Medium	Throttle agent; review sessions	T1119 / OWASP-LLM06
D11	**Injection indicator followed by side-effect tool call** Anomalous agent action	Injection flag on retrieval, then a side-effect tool call in the same session within 120 s	RAG + tool gateway	High	Force human approval or disable tool; review context	OWASP-LLM01
D12	**Log source went silent** Security-control failure	No heartbeat from a log source for 15 min or more	Pipeline heartbeat	High	Page on-call; check pipeline and source	T1562
D13	**Security control disabled or weakened** Security-control failure	Cloud audit event: MFA policy disabled, WAF detect-only, logging off, guardrail off	Cloud audit	Critical	Auto-revert; page Security; review change record	T1562
D14	**Secret detected in pushed commit** Credential use	Push with one or more secret-scanner findings	Git audit	High	Revoke key; run Part 4 playbook	T1552
D15	**PHI pattern found in application/prompt logs** Data protection failure	National-ID or MRN pattern with context keyword in app or prompt logs	App + prompt logs	High	Redact and purge per legal hold; fix logging code	OWASP-LLM02
D16	**LLM token spend spike (denial-of-wallet)** Cost / availability abuse	Tenant LLM tokens over 400,000 in a rolling hour	LLM egress gateway	Medium	Throttle tenant; notify owner; review prompts	OWASP-LLM10

10.3 Handling standards

- Every alert has an owner, a runbook link, and a response SLA: Critical pages on-call immediately; High within 1 hour; Medium next business day.
- Thresholds are hypotheses. Run in alert-only mode for two to four weeks on real baselines, then tune per actor before any automated response is enabled.
- Automation ladder: alert, then human-approved response, then automatic response only for high-confidence rules (D07, D13) after a clean tuning period.

- Quality metrics reported monthly: alerts per rule, false-positive rate, mean time to acknowledge, and a purple-team test that proves each rule still fires.

Part 11. Incident Response: Service Account Reads 25,000 Records in 12 Minutes

Bottom line. Treat this as a probable data breach until proven otherwise. In the first 15 minutes stop the bleeding with the **least disruptive step that works** (revoke the service account's credentials and block the external address) while keeping the workload intact for forensics. Then scope precisely, decide notifications with Legal and the DPO, eradicate, and recover. The decisions that matter most are made in the first hour.

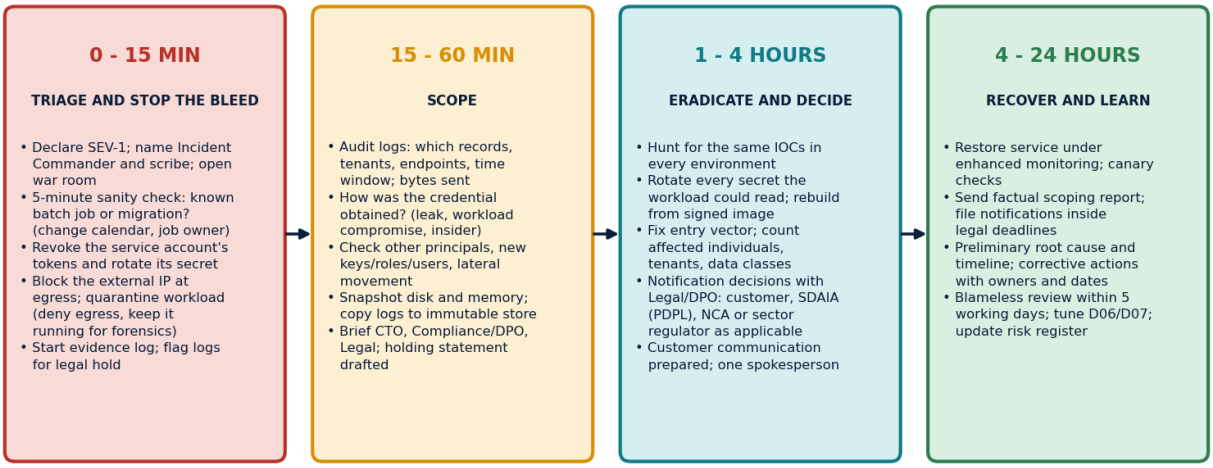


Figure 11.1. Actions by time window for the scenario in the brief.

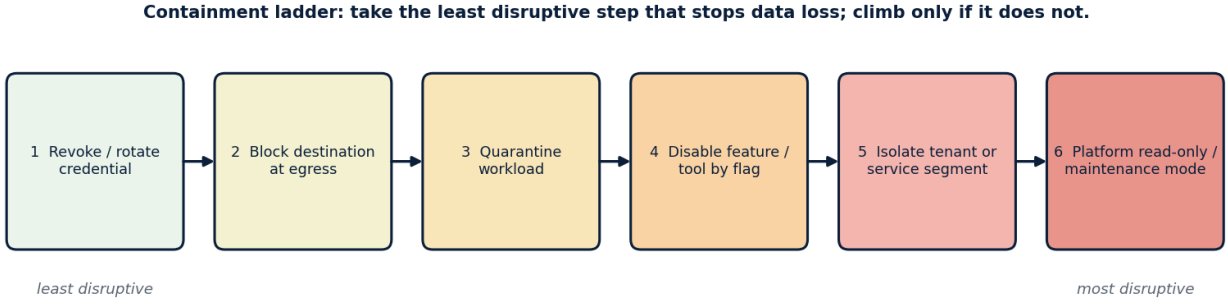


Figure 11.2. Containment ladder.

11.1 Roles and decisions

Role	Who	Owns
Incident Commander (IC)	Security Engineer or on-call senior	Declares severity, runs the timeline, makes containment calls, keeps one source of truth
Technical lead	DevOps or backend lead	Executes containment and eradication, preserves evidence
Scribe	Any engineer	Timestamped log of facts, decisions and who made them
Legal and DPO	Compliance/DPO with counsel	Notification decisions and wording; legal hold; regulator contact
Communications lead	CTO or delegate	Customer and internal messages; a single spokesperson

11.2 First-15-minute decision points

- **Is it real?** Five minutes maximum: check the change calendar and the job scheduler, and ask the service owner. A legitimate nightly export would explain volume but not the unknown destination.
- **Contain without an outage where possible:** revoke and rotate the credential and block the destination first. Quarantine the workload with deny-egress rather than deleting it, and never reboot before memory and disk snapshots.
- **Preserve evidence:** export logs to an immutable store; flag legal hold; write down what was done and when.
- **Do not** tip off an insider or announce externally before facts and wording are agreed.

11.3 Notification map

Audience	Trigger	Deadline to validate	Owner
Affected customer(s)	Any likelihood their patient data was accessed	Per contract (often 24-72 h); tell them early with facts, then update	CTO with Legal
SDAIA (PDPL competent authority)	Breach likely to harm individuals or their rights	72 hours from becoming aware [Validate]	DPO with Legal
NCA and sector regulators	As required for the entity and sector	Per applicable ECC and sector rules [Validate]	Compliance
Affected individuals	Severe harm likely	As required by law and by the customer, who is usually the controller [Validate]	Customer with Apex support
Cyber insurer, law enforcement	Per policy and counsel advice	Per policy	CTO

What good looks like. Detection rule D06 fires about 2 minutes after the first bulk read and D07 correlates the egress (Part 18). Even so, at 2 minutes of detection plus a 2-minute response, about 9,000 records could already be read. That is why the design also caps per-principal record budgets at the gateway (C11): detection limits loss, prevention bounds it.

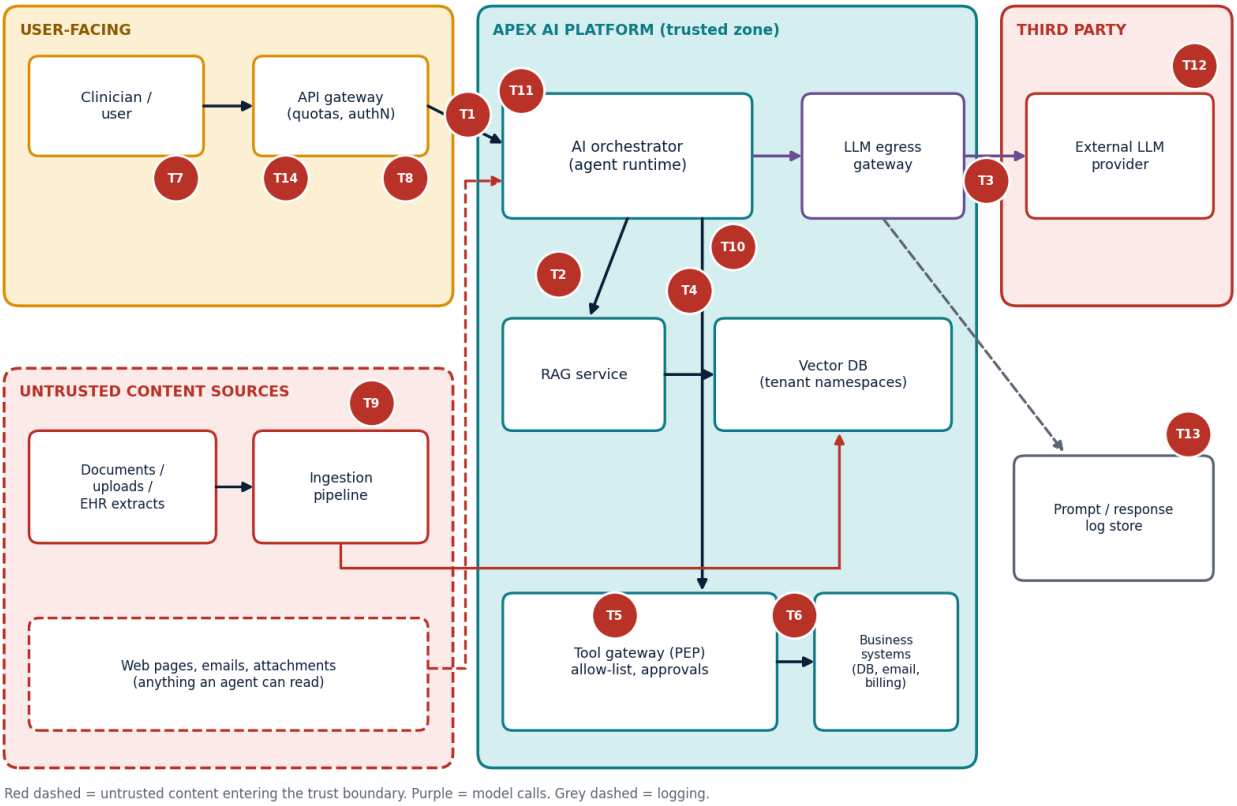
11.4 Success measures

- Time to declare (target under 15 min), time to contain (under 60 min), time to first customer contact (per contract), evidence completeness, and actions closed within 30 days.
- Blameless review within five working days; each finding becomes a detection, a control or a test, with an owner and a date.

Part 12. Threat Model for the AI Features

Bottom line. Prompt injection cannot be fully prevented, so the design assumes the model **will** sometimes be manipulated and makes sure that a manipulated model cannot reach data or actions it is not entitled to. Authorization, tenant isolation, approval gates and egress control sit outside the model. The 14 threats requested in the brief are mapped to the OWASP Top 10 for LLM Applications (2025 numbering) and to concrete controls and detections.

AI/LLM data-flow and threat map (T1-T14 refer to Table 12.1)



Red dashed = untrusted content entering the trust boundary. Purple = model calls. Grey dashed = logging.

Figure 12.1. Data-flow diagram with the 14 threat locations.

Table 12.1. Threat register for AI features (14 threats)

ID	Threat	Attack path in Apex	OWASP LLM	L/I	Preventive controls	Detective	Residual
T1	**Direct prompt injection**	User types instructions to override rules or extract data (for example asking the assistant to list all patients)	LLM01	4/4	Separate system and user roles; enforce policy outside the model (retrieval filters, tool gateway); no secrets in prompts; input length caps. Refusal wording is not the control.	D09-D11; injection heuristics logged	Medium
T2	**Indirect prompt injection**	Hidden instructions in an uploaded document, email or web page that RAG or an agent reads	LLM01	4/5	Mark retrieved content as untrusted data; injection scan at ingestion; taint rule lowers agent autonomy; no tool call justified only by retrieved text; egress allow-list; block auto-loaded remote content	D11; retrieval flags	Medium-High
T3	**Sensitive data leakage**	Model reveals PHI from context or history, or the system prompt	LLM02 LLM07	3/5	Minimum-necessary context; PHI tokenization; no secrets in prompts; output DLP; per-user context only	D15; output DLP alerts	Medium
T4	**Unauthorised retrieval**	RAG returns chunks the user may not see (other patient or tenant)	LLM02 LLM08	4/5	Tenant and ACL pre-filter inside the vector query; policy re-check per chunk; per-tenant namespaces; fail closed; canary tests	Canary alarms; retrieval audit	Low after fix
T5	**Excessive agent permissions**	Agent holds broad credentials so a successful injection inherits all of it	LLM06	4/5	Per-agent identity; on-behalf-of scoped short-lived tokens; tool allow-list; prohibited tools; no ambient cloud credentials	D09 D10	Low-Medium
T6	**Tool misuse and unsafe actions**	Agent sends an email, refund or update with attacker-chosen parameters	LLM06	3/5	Human approval bound to payload hash; parameter constraints; amount limits; idempotency keys; dry-run	D09; approval audit	Low-Medium
T7	**Hallucination and unsafe clinical content**	Model states something false with confidence and a clinician relies on it	LLM09	4/4	Position as decision support with human review; answers require citations (no source, no answer); uncertainty display; scope limits; clinical safety review; feedback loop. Whether medical-device rules apply needs confirmation [Validate]	User-flagged errors; sampled review	Medium (cannot be eliminated)
T8	**Insecure output handling**	Model output rendered as HTML or markdown, or passed into SQL or a shell	LLM05	3/4	Treat output as untrusted: encode and sanitise, strict CSP, no automatic remote images or links, parameterised queries only, code only in a sandbox	CSP reports; WAF	Low
T9	**Poisoned RAG documents**	Attacker uploads or edits documents to bias answers or plant instructions	LLM04 LLM08	3/4	Authenticated ingestion; source allow-list; content scanning; provenance and versioning; quarantine external sources; review for shared corpora	Ingestion anomaly alerts; hash changes	Medium
T10	**Credential exposure**	Keys in prompts, tool outputs, logs or agent memory; model echoes a secret	LLM02	3/5	Secrets only in the gateway and vault; redact secrets in tool outputs; secret scanning of logs and prompts; no credentials in context	D14; secret patterns in logs	Low
T11	**Cross-user and cross-tenant leakage**	Shared cache, conversation memory, embeddings or fine-tuning mixes users' data	LLM02	3/5	Per-user and per-tenant session isolation; cache key includes tenant and ACL hash; no shared semantic cache; no tenant data used for training without consent	Canary tests; cache audit	Low
T12	**Third-party model risk**	Provider retains or trains on data, changes a model, has a breach, or processes outside approved regions	LLM03	4/4	DPA, zero retention, no training, region and subprocessor terms [Validate]; pinned model versions; regression and red-team tests on upgrade; fallback provider and kill switch	Egress logs; D08; provider notices	Medium
T13	**Sensitive prompt logs**	Prompts and responses with PHI stored in a broadly readable log store	LLM02	5/4	Metadata-only logging; redaction at source; separate restricted store with short TTL and JIT access	D15	Low
T14	**Denial-of-wallet and exhaustion**	Huge prompts, loops or many requests run up cost or exhaust quota	LLM10	4/3	Token and step budgets; per-tenant quotas; timeouts; circuit breakers; maximum tool steps	D16; cost anomaly alerts	Low

L/I = likelihood and impact on the Part 1 scales before the proposed controls. Residual is my judgement after the controls. OWASP LLM identifiers follow the 2025 list (LLM01 Prompt Injection to LLM10 Unbounded Consumption).

Part 13. Secure Retrieval-Augmented Generation (RAG)

Bottom line. Retrieval must be **authorization-aware**: the vector search itself is restricted to what the user may read, a policy engine re-checks each chunk against the source of truth, and any failure to determine access means "deny". The model only ever sees chunks the user is entitled to, so even a successful prompt injection cannot make it reveal what it never received.

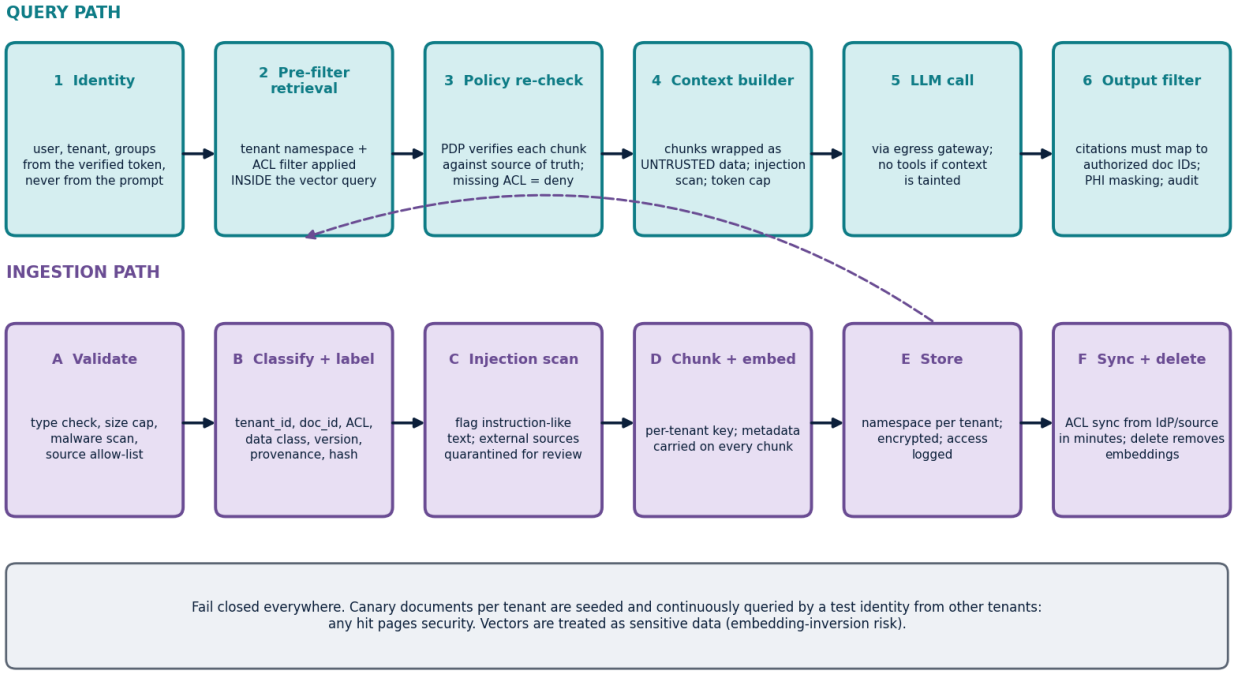


Figure 13.1. Query path (top) and ingestion path (bottom).

13.1 Design choices that matter

Choice	Decision	Reason
Where identity comes from	Verified token: user, tenant, groups, clearance	Anything the user types is untrusted; identity cannot be claimed in a prompt
Pre-filter versus post-filter	Filter **inside** the vector query (tenant namespace plus ACL metadata), then re-check with the policy engine	Post-filtering alone wastes the top-k slots on forbidden chunks (poor answers) and risks leaks through ranking or timing
Tenant isolation	Namespace per tenant; optional per-tenant encryption key for high-sensitivity tenants	Hard boundary that is easy to evidence to a customer
Stale permissions	ACL sync from the source in minutes, and a **query-time** policy check	Revocation must take effect immediately, not at the next sync
Missing or unknown ACL	Deny (fail closed)	Ingestion bugs must not become disclosures
Caching	Cache key includes tenant and ACL hash; no shared semantic cache across users	Cross-user leakage through cache hits
Context assembly	Wrap chunks as untrusted data; injection scan; token cap; no instructions accepted from context	Indirect prompt injection
Output	Citations must map to authorised document IDs; PHI masking; audit of IDs, never content	Detects leakage and supports audits
Vectors as data	Encrypt, restrict access, include in erasure and retention	Embeddings can be partially inverted to text
Deletion	Deleting a document removes chunks and embeddings within an SLA; backups expire by schedule	Data-subject rights and retention duties [Validate]

13.2 Chunk metadata carried on every record

```
{ "tenant_id": "t-104", "doc_id": "d-8841", "chunk_id": "d-8841#12",
  "acl_any": ["group:cardiology-clinicians", "user:u-2291"],   # principals allowed to read
  "data_class": "C3", "source": "upload|ehr|kb", "version": 7,
  "content_hash": "sha256:...", "ingested_at": "2026-09-18T09:14:00Z",
  "provenance": {"submitter": "u-2291", "scan": "clean", "injection_score": 0.02} }
```

13.3 Retrieval sketch ([Design], not executed)

```
def retrieve(token, query, k=8):
    ctx = verify(token)                                  # tenant, user, groups from a signed token
    hits = vector_db.search(embed(query), top_k=k * 3,
                            namespace=f"tenant:{ctx.tenant_id}",           # hard isolation
                            filter={"acl_any": ctx.principals, "class_max": ctx.clearance})
    allowed = []
    for h in hits:
        d = pdp.check(ctx, "read", h.doc_id)             # source-of-truth re-check
        audit.log(ctx.user_id, h.doc_id, d.allow)        # IDs and decision only
        if d.allow:                                      # unknown or missing ACL => deny
            allowed.append(h)
        if len(allowed) == k:
            break
    return wrap_as_untrusted(allowed)
```

13.4 Tests that prove it (run in the pipeline and continuously)

Test	Expected result
Tenant B identity searches for a seeded canary phrase that exists only in tenant A	Zero chunks returned; any hit pages Security
User without the group requests a document in the same tenant	Zero chunks; decision logged
ACL revoked, query issued within one minute	Denied (query-time check), even before sync completes
Document deleted	Not retrievable; embeddings removed within the SLA
Document containing instruction-like text is ingested	Flagged and quarantined; not used until reviewed
Chunk with missing ACL metadata	Never returned
Same query from two different users	Separate cache entries; no cross-hit
Answer cites a document	The document ID is in the user's authorised set

Part 14. Tool Permissions for the AI Agent

Bottom line. Classify by **what the worst case does**, not by how useful the tool is. Read-only, entitlement-checked lookups can run autonomously; anything that changes data or leaves the organisation needs a human to approve the exact payload; anything that creates privilege or destroys data is not available to an agent at all.

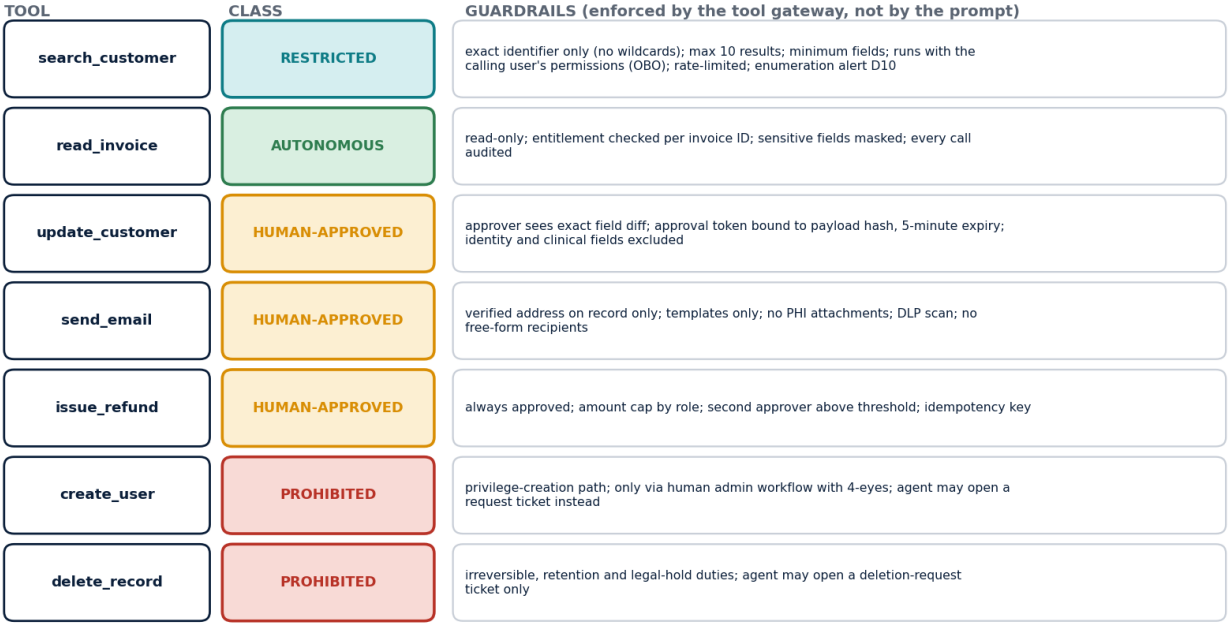


Figure 14.1. Classification of the seven agent tools and the guardrails on each.

14.1 Why each classification

Tool	Class	Reasoning
search_customer	Restricted	Reads personal data and is the natural enumeration tool. Allowed automatically only with an exact identifier, small result sets, the user's own permissions and rate limits.
read_invoice	Autonomous	Read-only and scoped to an ID already inside the user's entitlement; low blast radius; fully audited.
update_customer	Human-approved	Changes records that other systems and clinicians rely on; an injected instruction could alter data silently.
send_email	Human-approved	An outbound channel: phishing, disclosure and data exfiltration in one tool. Verified recipient and template only.
issue_refund	Human-approved	Moves money. Approval always; a second approver above a threshold.
create_user	Prohibited	Privilege creation is the strongest attacker goal; only a human admin workflow with two-person control.
delete_record	Prohibited	Irreversible and subject to retention and legal-hold duties; the agent may only open a request ticket.

14.2 Enforcement: the tool gateway, not the prompt

- **Policy enforcement point** in front of every tool. Schemas validate parameters; the policy engine checks class, tenant, user entitlement and limits; the model cannot bypass it.
- **On-behalf-of tokens:** each call carries a short-lived token derived from the calling user and narrowed to that one tool and resource. The agent holds no standing credential.
- **Approval design:** the approver sees the exact field diff or message; the approval token is bound to the payload hash and expires in about five minutes, so the model cannot change the payload after approval.
- **Taint rule:** if the context includes retrieved or external content, autonomy drops one level. Restricted becomes human-approved; prohibited stays prohibited.
- **Budgets:** maximum steps, time and tool calls per session; per-tenant spend limits; idempotency keys to stop repeats.
- **Kill switches:** per-tool and per-agent flags that operations can flip in seconds, with alerts D09-D11 as triggers.
- **Change control:** a new tool or a class change needs security review, tests in the pipeline and a documented rollback.

14.3 How I would test it

- A matrix test for every tool and class: unapproved calls, replayed approvals, altered payloads, and calls from tainted context must all be denied.
- Indirect-injection cases embedded in documents and emails, run in staging before each release; results kept as evidence.
- Quarterly red-team exercise focused on agent abuse, with findings entering the Part 8 SLA.

Part 15. Data Protection Design

Bottom line. Classify data once, and let the class decide handling everywhere: storage, access, logging, non-production use, and whether it may ever reach an external model. Patient data (class C3) is minimised, tokenized before any model call, never logged as content, and not sent to an external provider unless the provider, region and contract have been approved. Secrets (class C4) never enter prompts at all.

15.1 Classification and handling

Class	Examples	Storage and access	Logging	External LLM	Non-production
C0 Public	Marketing content, public documentation	No special control	Normal	Allowed	Allowed
C1 Internal	Runbooks, internal tickets without personal data	Staff access by role	Normal	Approved providers	Allowed
C2 Confidential	Customer contracts, source code, non-PHI customer data, architecture	Need-to-know; encryption at rest; access logged	Metadata and object IDs	Approved providers, minimised	Only if masked
C3 Sensitive personal and health data	Patient records, national ID, MRN, clinical notes and images, prompts containing them	Tenant-isolated; CMK encryption; JIT access; field-level encryption for identifiers; masked by default	IDs and decisions only; never content	**Blocked** unless tokenized, or approved in-Kingdom provider and customer agree [Validate]	**Never**; synthetic or de-identified only
C4 Secrets	API keys, tokens, private keys, passwords	Secrets manager only; short-lived; never in code, images or prompts	Never logged; scanned for	**Never**	Separate secrets per environment

Mapping to the customer's or national classification scheme (for example the four-level public/restricted/secret/top-secret scheme used in Saudi government data governance) must be agreed with each customer. [Validate]

15.2 What may reach an external AI provider

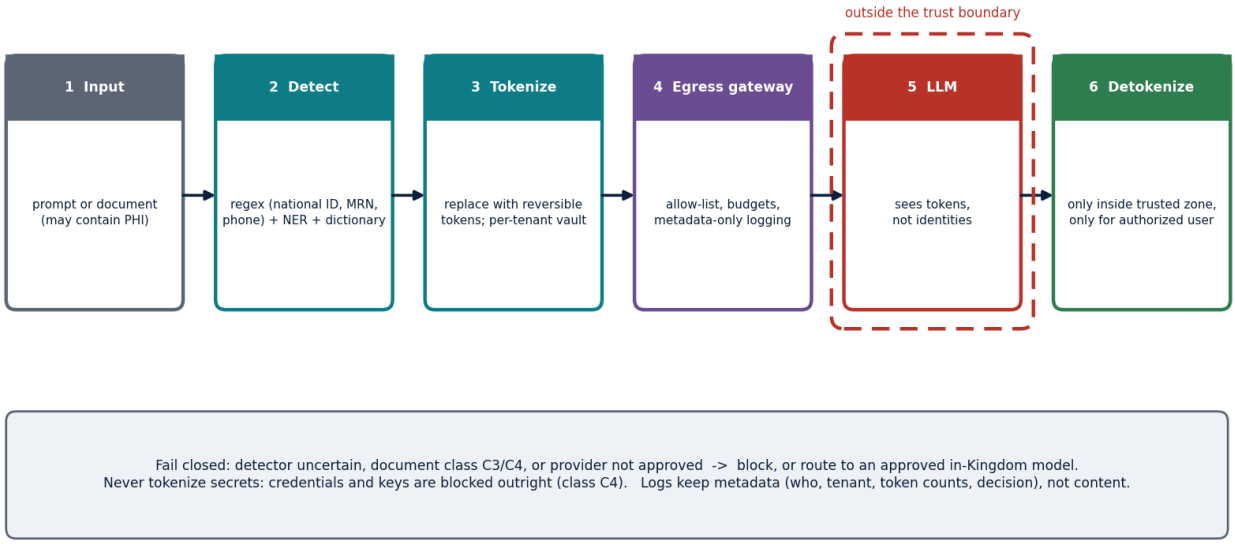


Figure 15.1. PHI-safe path to an external model. The model sees tokens; identities are restored only in the trusted zone for authorised users.

- **Minimisation:** send only the fields the task needs. Free-text notes are the highest risk and are tokenized or excluded.
- **Detection:** layered regexes (national ID, MRN, phone, email), named-entity recognition, and dictionaries. Detection is probabilistic, so it fails closed for C3 documents.
- **Provider terms to require [Validate]:** no training on Apex or customer data, zero or minimal retention, region and residency commitments, subprocessor list, breach-notification time, audit rights, and deletion on request.
- **Alternative for the most sensitive flows:** an approved model hosted inside the Kingdom or in Apex's own environment, chosen after the residency review.

15.3 Retention and deletion (proposal)

Data	Proposed retention	Notes
Raw prompts and responses	Not stored by default. If needed for quality: redacted, restricted store, maximum 30 days	Metadata (tenant, counts, decision) kept 12 months
Security and audit logs	12 months searchable, then archive per policy	Confirm against the applicable ECC logging control and contracts [Validate]
Patient and customer records	Per customer contract and applicable health-record rules	Apex acts on the customer's instructions; deletion must propagate to indexes and caches [Validate]
Embeddings and caches	Same lifecycle as the source document	Deleted with the source; included in erasure
Backups	Rolling 35 days plus monthly snapshots for up to 12 months	Deletion completes on expiry; the customer is told this [Validate]
Incident records	Per policy and legal advice	Under legal hold while an investigation is open

15.4 Encryption, keys and masking

- In transit: TLS 1.2+ (1.3 preferred) externally; mutual TLS between services; certificate verification always on.
- At rest: AES-256 through the cloud key service using customer-managed keys; envelope encryption with per-tenant keys for sensitive tenants; field-level encryption for national ID and MRN.
- Key management: HSM-backed root, separate key administrators from data administrators, annual rotation and rotation on personnel change, decrypt events logged and alerted.
- Masking: national ID shown as last three digits; names and dates of birth masked in admin tools and logs; unmask needs a reason and is audited.
- Backups, replicas, exports and embeddings are covered by the same encryption and access rules.

Part 16. Mapping to Saudi Requirements and Recognised Frameworks

Bottom line. The design maps cleanly onto the NCA Essential Cybersecurity Controls, cloud and data controls, and PDPL principles, and onto ISO 27001:2022, NIST CSF 2.0 and CIS Controls v8. The mapping is at **subdomain and topic level**. Article and control-level citations, the applicable ECC edition (ECC-1:2018 or ECC-2:2024), and whether Apex is directly in scope or in scope through customers must be confirmed by Compliance and Legal before this is used as audit evidence. [Validate]

Control area in this design	NCA ECC subdomain	NCA CCC / DCC	PDPL topic	ISO 27001:2022 Annex A	NIST CSF 2.0	CIS v8	Part
Governance, risk register, policies, roles	1-1 to 1-5, 1-7, 1-8	Governance domains	Accountability and records	Clauses 4-10; A.5.1-5.4	GV.RM, GV.PO, GV.RR	n/a	1, 19
Asset inventory and ownership	2-1 Asset management	CCC asset controls	Records of processing	A.5.9	ID.AM	1, 2	1, 16
Identity, MFA, privileged and JIT access	2-2 Identity and access management	CCC IAM	Security safeguards	A.5.15-5.18, A.8.2, A.8.5	PR.AA	5, 6	3, 5
Network segmentation, edge, egress control	2-4 Network security	CCC network	Security safeguards	A.8.20-8.22	PR.IR	12, 13	2
Server, endpoint and container hardening	2-3 Systems protection; 2-5 Mobile devices	CCC configuration	Security safeguards	A.8.9, A.8.1	PR.PS	4	6, 17
Data classification, minimisation, masking	2-6 Data and information protection	DCC (data controls by class)	Purpose limitation, minimisation, sensitive (health) data	A.5.12-5.13, A.8.11	PR.DS	3	15
Encryption and key management	2-7 Cryptography	DCC, CCC cryptography	Security safeguards	A.8.24	PR.DS	3	15
Backup, recovery, resilience	2-8 Backup; 3-1 Resilience	CCC resilience	Availability and integrity of personal data	A.8.13, A.5.29-5.30	PR.DS, RC.RP	11	1, 20
Vulnerability management and SLA	2-9 Vulnerabilities	CCC vulnerability	Security safeguards	A.8.8	ID.RA, PR.PS	7	8
Penetration testing and AI red-team	2-10 Penetration testing	CCC testing	Security safeguards	A.8.8, A.8.29	ID.IM	18	1, 14
Logging, monitoring, detection	2-11 Event logs and monitoring	CCC logging	Breach detection	A.8.15, A.8.16	DE.CM, DE.AE	8, 13	10, 18
Incident response and notification	2-12 Incident and threat management	CCC incident	Breach notification to authority and individuals	A.5.24-5.28	RS.MA, RS.CO	17	11
Web, API and application security	2-14 Web application security	CCC application	Security safeguards	A.8.26, A.8.28	PR.PS	16	7, 13
Secure SDLC, CI/CD, change control	1-6 Cybersecurity in IT projects; 2-3	CCC change	Privacy and security by design	A.8.25-8.29, A.8.32	PR.PS, GV.SC	16	9
Third parties and AI providers	4-1 Third-party cybersecurity	CCC third party	Processor contracts and cross-border transfer	A.5.19-5.23	GV.SC	15	12, 15
Cloud hosting, residency, tenant controls	4-2 Cloud computing and hosting	CCC (tenant and provider)	Cross-border transfer; residency	A.5.23	GV.SC, PR.IR	12, 15	2, 15
Awareness and training (incl. AI misuse)	1-10 Awareness and training	n/a	Accountability	A.6.3	PR.AT	14	9, 19
Data-subject rights, retention, deletion	2-6 (data lifecycle)	DCC disposal	Access, correction, destruction; retention	A.5.34, A.8.10	PR.DS	3	13, 15
AI-specific risks (LLM and agents)	No dedicated control; apply 2-14 and 2-6	No dedicated control	Automated processing; sensitive data	A.8.26-8.28 (with ISO/IEC 42001 as guide)	GV, ID.RA; NIST AI RMF as guide	16	12-14

Open validation items: (1) whether Apex is a direct ECC/CCC/DCC subject or through customer contracts; (2) ECC edition and control numbering; (3) PDPL article references, controller/processor roles and cross-border transfer conditions for external LLM use; (4) health-sector requirements and any medical-device classification of clinical features; (5) log retention periods; (6) breach-notification deadlines and channels; (7) data residency and any cloud-provider registration requirements. Owner: Compliance/DPO with external counsel; target: within 30 days, feeding the Day-60 compliance validation in the roadmap.

Part 17. Linux Server Hardening: Top 20 Checks

Bottom line. Prefer immutable, replaced-not-patched hosts with no SSH at all. Where hosts persist, these 20 checks cover patching, access, network exposure, kernel and filesystem settings, auditing, container runtime and secrets. Each has a command and a pass criterion so a review produces evidence, not opinion. [Design]; commands are standard Linux tools and were not run against a real host.

#	Check	Why	Verify with	Pass
1	Patch level and automatic security updates	Known kernel and package flaws are the commonest host entry	`apt list --upgradable | wc -l; systemctl is-enabled unattended-upgrades`	No pending security updates, or image replaced weekly
2	SSH: no root login, no passwords; ideally no SSH (use session manager or ZTNA)	Brute force and stolen passwords	`sshd -T | grep -Ei 'permitrootlogin|passwordauthentication'`	no / no
3	Named accounts; only root has UID 0; sudo without NOPASSWD	Attribution and least privilege	`awk -F: '$3==0' /etc/passwd; grep -R NOPASSWD /etc/sudoers /etc/sudoers.d`	Only root; none (or audited automation)
4	No empty passwords; unused accounts locked	Trivial access	`awk -F: '($2==""){print $1}' /etc/shadow`	No output
5	Only required services enabled	Fewer moving parts	`systemctl list-unit-files --state=enabled`	Matches documented baseline
6	Listening ports match the design	Unexpected listeners are exposure	`ss -tulpn`	Only expected ports; none public except through the load balancer
7	Host firewall default deny	Second layer behind cloud rules	`nft list ruleset | head -40`	Input policy drop; allow-list documented
8	Kernel hardening sysctls	Reduces exploit primitives	`sysctl kernel.kptr_restrict kernel.dmesg_restrict kernel.unprivileged_bpf_disabled net.ipv4.conf.all.rp_filter net.ipv4.tcp_syncookies kernel.randomize_va_space`	2, 1, 1 or 2, 1, 1, 2
9	Disk encryption with managed keys	Lost or copied volumes	`lsblk -f`	Encrypted, key under customer-managed control
10	Restrictive mount options on /tmp, /var/tmp, /dev/shm	Blocks executing dropped files	`findmnt -no OPTIONS /tmp /dev/shm`	nodev,nosuid,noexec
11	No world-writable files without sticky bit	Tampering and privilege escalation	`find / -xdev -type f -perm -0002 | head`	None
12	SUID/SGID inventory	Classic escalation route	`find / -xdev -perm /6000 -type f`	Matches approved baseline
13	auditd rules for identity files, sudo, sshd config, docker socket	Forensic trail	`auditctl -l | head -30`	Rules loaded and forwarded to the SIEM
14	Time sync and log forwarding	Correlation and evidence	`chronyc tracking; systemctl is-active rsyslog`	Synchronised; forwarding active
15	Docker daemon hardened: no exposed TCP socket, no docker.sock mounts	Socket access equals root	`ss -tlnp | grep -E ':237[56]'; docker info --format '{{.SecurityOptions}}'`	No listener; seccomp enabled
16	Mandatory access control enforcing	Confines compromised processes	`aa-status | head -5 (or getenforce)`	Enforcing
17	Containers unprivileged and non-root	Ties to Part 6	`docker inspect $(docker ps -q) --format '{{.Name}} {{.HostConfig.Privileged}} {{.Config.User}}'`	false and a non-root user
18	No plaintext secrets on disk	Backups and images copy them	`grep -RIlE 'PRIVATE KEY|api[_-]?key|secret' /etc /home /opt | head`	None; secrets manager only
19	Cloud metadata service protected; instance role least-privilege	SSRF to credentials	`provider-specific: metadata requests must need a session token; review attached role`	Token required; minimal role
20	Baseline scan, drift and runtime detection	Continuous assurance	`lynis audit system --quick (or OpenSCAP with a CIS profile); systemctl is-active <edr-agent>`	Score tracked; exceptions recorded; agent active

Part 18. Hands-On Exercise: Detection Engineering (Executed Evidence)

Bottom line. I chose detection engineering because the brief's central incident (25,000 records in 12 minutes, then an unknown external IP) is a detection and response problem. I built 16 detection rules, a synthetic log generator with 11 labelled attacks and deliberate benign near-misses, 50 unit tests, a mutation check, and an evaluation harness, and I ran all of it. On synthetic data: **0 false alerts** on the benign day, **11 of 11** attack scenarios detected with no unexpected alerts, and the brief's incident flagged **130 seconds** after the first bulk read. These results show the implementation behaves as designed; they do **not** estimate production accuracy.

18.1 What was built and run [Executed]

Artefact	What it is	Result
engine.py, config.json	16 rules, pure Python, thresholds in a config file, alerts carry ATT&CK or OWASP LLM mapping and never contain raw patient identifiers	Runs on JSON-lines events
generate_logs.py	Seeded (42) synthetic logs: benign day 12,050 events; attack day 12,209 events; 11 scenarios; RFC 5737 documentation IPs; fake identifiers	Reproducible
test_rules.py	Positive, negative and boundary tests for every rule, plus 3 end-to-end tests	**50 of 50 pass**
mutation_check.py	Injects 12 deliberate bugs into the engine and confirms the tests fail	**12 of 12 killed**. First run killed 11: it exposed an untested condition in D01 (5-minute window); I added a test and re-ran
run_eval.py	Scores scenarios, measures time to detect, models exposure in the brief's incident, and runs a threshold sensitivity study	Outputs in results/

18.2 Benign near-misses deliberately included

To test that rules are not just noisy tripwires, the benign day includes activity that resembles attacks: a clinician reading 45 records in 10 minutes; a developer failing a password three times; an approved privileged grant with a change ticket; onboarding three users in ten minutes; a legitimate batch job reading up to about 2,700 records per 10 minutes; a service pod restarting with a new IP in the same subnet; an agent making 12 calls in five minutes; an injection flag with no follow-up action; an approved refund; 10-digit numbers in logs with no patient context; and a WAF rule update that is not a control being disabled. None raised an alert.

18.3 Scenario results

ID	Scenario	Expected rules	Detected	Time to all expected alerts
S1	Admin credential attack, success without MFA	D01, D02	✓	240 s
S2	Privilege escalation and backdoor accounts	D03, D04	✓	228 s
S3	Service credential used from unseen network	D05	✓	0 s
S4	Service account reads 25,000 records in 12 min, then external egress	D06, D07, D08	✓	960 s
S5	Agent executes unapproved/prohibited tools	D09	✓	0 s
S6	Agent enumerates customers	D10	✓	80 s
S7	Indirect injection then side-effect call	D11	✓	40 s
S8	WAF weakened and gateway logging silent	D12, D13	✓	840 s
S9	Secret committed	D14	✓	0 s
S10	PHI written to prompt logs	D15	✓	0 s
S11	Token spike (denial-of-wallet)	D16	✓	1,020 s

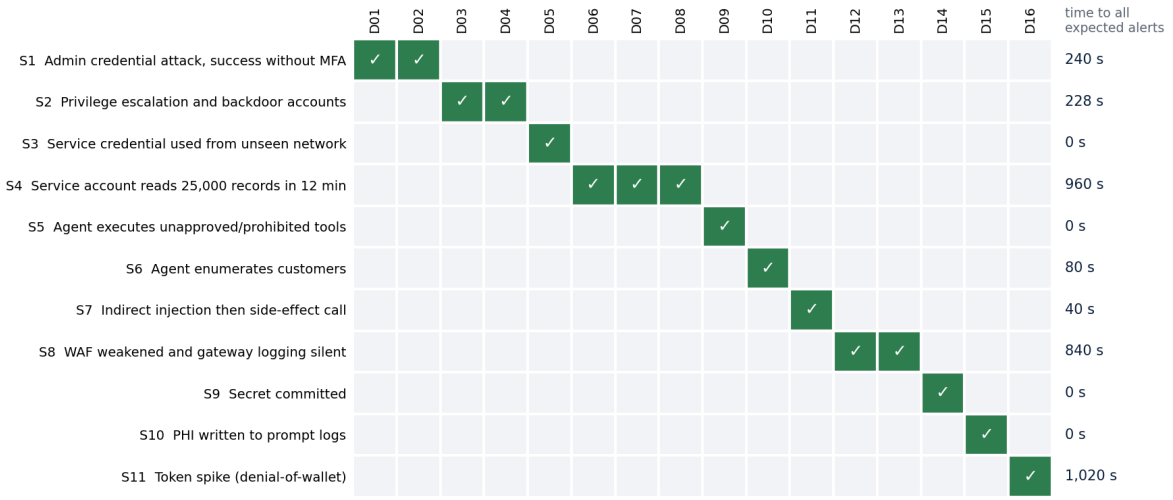


Figure 18.1. Scenario-by-rule coverage. Single-event rules alert at 0 s; correlation and volume rules need time to accumulate evidence.

18.4 The brief's incident replayed (scenario S4)

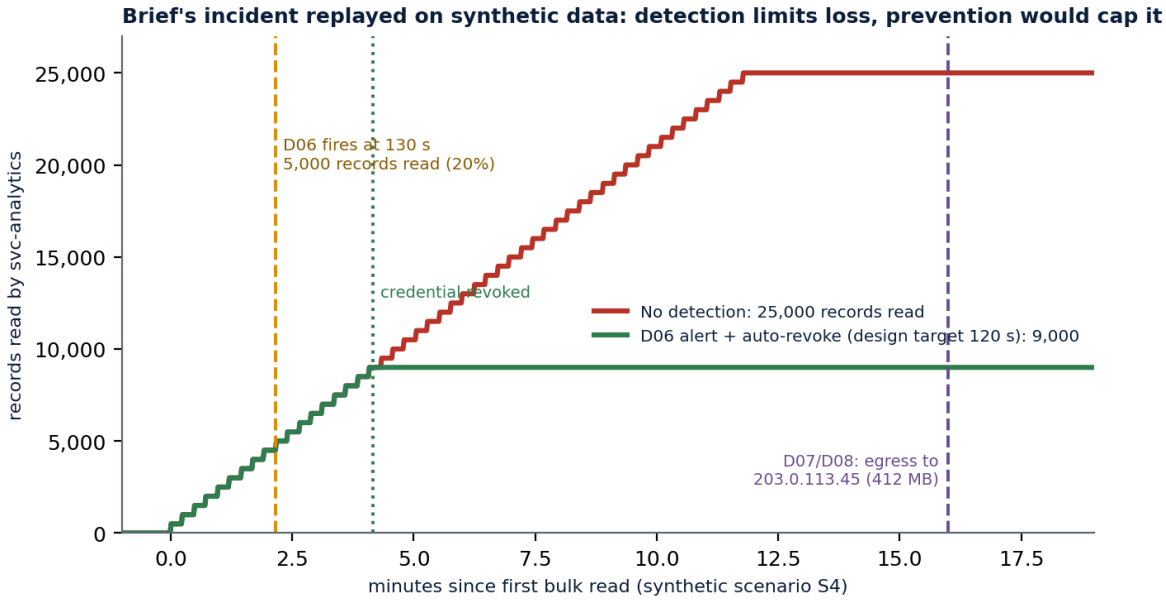


Figure 18.2. 50 calls of 500 records each over about 12 minutes, then 412 MB to an external address (synthetic).

Time (UTC, synthetic day)	Alert	Detail
15:22:09	**D06** High	5138 records in 10 min (threshold 5000)
15:36:00	**D07** Critical	mass read then 412000000 bytes to 203.0.113.45
15:36:00	**D08** High	dst 203.0.113.45:443 bytes=412000000

What this shows. D06 crossed its 5,000-record threshold 130 seconds in, when 5,000 of 25,000 records (20%) had been read. If an automatic credential revoke takes a further 120 seconds (a design target I have **not** measured), 9,000 records (36%) are exposed. Detection therefore limits the loss but does not prevent it. That is why Part 7 adds preventive per-principal record budgets (C11): a cap at the gateway bounds the loss regardless of how fast anyone reacts.

18.5 Threshold trade-off

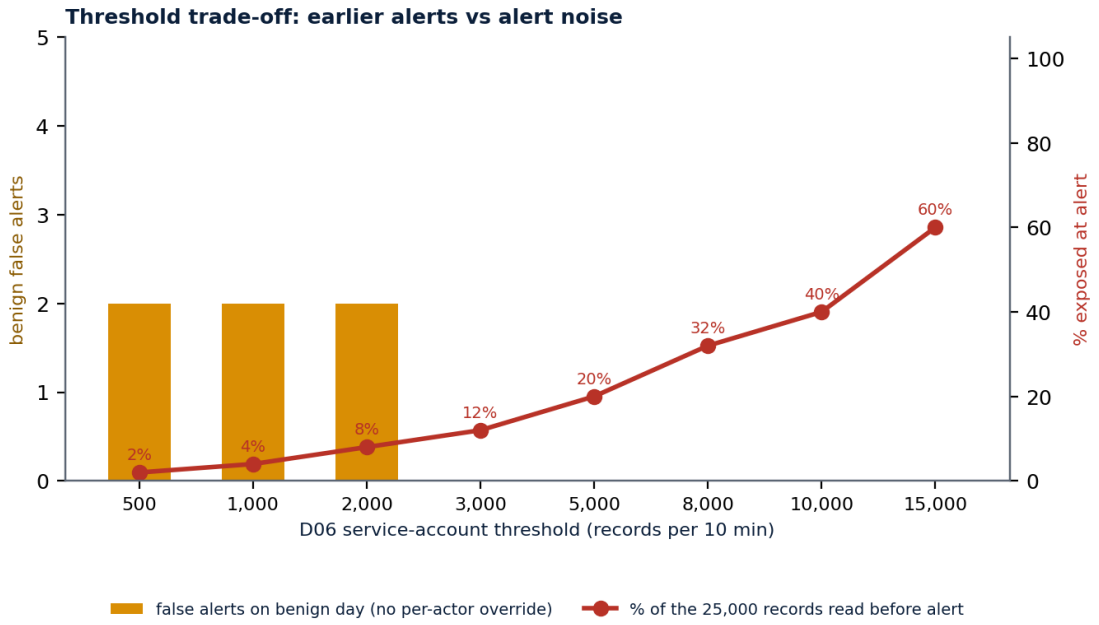


Figure 18.3. Effect of the service-account threshold on D06 (per-actor override removed to show the effect).

A lower threshold catches the incident earlier (12% of records at 3,000; 2% at 500) but alerts on the legitimate batch job at 2,000 or lower unless that actor has its own override. I set the default at a conservative 5,000 for week one, with a 3,000 override for the batch account, because real baselines are unknown. The next step is per-actor thresholds set from two to four weeks of observed behaviour (for example a multiple of each actor's 99th percentile), which would move the default toward the lower end without the noise.

18.6 The core rule (verbatim from the evidence folder)

```
def d06_mass_record_access(events, cfg):
    c = cfg["mass_read"]
    win = timedelta(minutes=c["window_min"])
    cool = timedelta(minutes=c["cooldown_min"])
    by_actor = defaultdict(list)
    for e in events:
        if e["source"] == "api" and e["event"] == "record_read":
            by_actor[e["actor"]].append(e)
    out = []
    for actor, evs in by_actor.items():
        atype = evs[0].get("actor_type", "human")
        thr = c["per_actor_override"].get(actor, c["service_threshold"] if atype != "human" else c["human_threshold"])
        dq: deque = deque()
        total = 0
        last_alert = None
        for e in evs:
            dq.append((e["_ts"], e.get("records", 1)))
            total += e.get("records", 1)
            while e["_ts"] - dq[0][0] > win:
                total -= dq.popleft()[1]
            if total > thr and (last_alert is None or e["_ts"] - last_alert > cool):
                out.append(_alert("D06", "Mass record access", "High", e["_ts"], actor,
                                  f"{total} records in {c['window_min']} min (threshold {thr})", "T1213",
                                  records_in_window=total, threshold=thr))
                last_alert = e["_ts"]
    return out
```

18.7 Limits and what I would do next

- **Synthetic and self-authored.** The same person wrote the data and the rules, so a clean result is partly self-fulfilling. Real logs will contain patterns I did not imagine.
- **Batch, not streaming.** A SIEM would run these as streaming or scheduled queries. Alert time here is the event time that first met the condition.
- **Static baselines.** D05 learns a /24 baseline from the first 12 hours; production needs rolling baselines and onboarding for new services. D15 uses context-keyword patterns; national-ID checksum validation should be added after confirming the official algorithm.
- **Next:** translate the 16 rules into the chosen SIEM's query language; run in alert-only mode for two to four weeks; tune per actor; run a purple-team exercise that replays S1-S11 against real telemetry; only then enable automated responses for D07 and D13.

Reproduce

```
cd detection_engineering
python generate_logs.py            # synthetic data, seed 42
python -m unittest -v test_rules   # 50 tests
python mutation_check.py           # 12 injected bugs must all fail the tests
python run_eval.py                 # writes results/*
```

Part 19. Responding to the Executive: "We use HTTPS, a major cloud and a managed database, so we are secure enough."

19.1 The 30-second answer

> *"Not yet, but we are close to fixing the important parts. HTTPS, a major cloud and a managed database are good foundations. They protect the connection and the hardware. They do not protect our logins, our settings or our AI features, and that is where patient data is actually at risk. Our own review found 19 gaps, eight rated critical: no multi-factor login on admin access, a shared admin account, a database reachable from developer laptops, patient data in prompt logs, and AI agents that can do more than they should. Any one of these can become a patient-data incident and a lost customer. Most of the top fixes take days, not months. I recommend we run the demo on synthetic data now and hold real patient data until ten evidence items are green."*

19.2 Why the statement is incomplete, in business terms

The assumption	What it covers	What it does not cover (from our findings)	Business consequence
We use HTTPS	Protects data in transit between browser and server	Stolen or guessed logins (#3, #2), flawed authorization, AI prompt attacks, patient data written to logs (#8)	A breach can happen through a valid, encrypted connection
We use a major cloud	The provider secures the physical and platform layers (shared responsibility)	Our identities, firewall rules, keys and configuration are ours: shared admin (#2), open database rule (#7), keys on laptops (#4)	Most cloud breaches come from customer-side configuration
We use a managed database	Provider patches and runs the database engine	Who is allowed to connect and what they can read; backups actually restorable (#10)	A managed database with an open door is still open
Therefore secure enough	Basic hygiene	Healthcare customers, PDPL duties and procurement reviews ask for evidence, not assurances	Lost deals, contract penalties, regulator scrutiny, reputational damage

Regulatory exposure exists under the Personal Data Protection Law for personal health data; the exact penalties and Apex's role as processor or controller need Legal's confirmation. [Validate] I would not quote numbers to the executive that I cannot support.

19.3 Minimum evidence before approving production use with patient data

#	Evidence item	Where it comes from	Pass criterion	Owner
1	MFA enforced for all privileged and staff accounts	IdP report; failed no-MFA login test	100% of privileged users; no bypass path	IT Lead
2	No shared admin; no static keys older than 90 days	Cloud credential report	Zero shared credentials; zero stale keys	DevOps
3	Database and admin not reachable from the internet or developer IPs	Rule export; external and internal scan	Ports closed; rule diff approved	DevOps
4	Backup restore test with measured RTO and RPO	Restore job log	Restore succeeds within agreed targets	DevOps
5	Authenticated API test on the four endpoints incl. cross-tenant tests	Test report (internal or external)	No critical or high open; BOLA tests all pass	Security
6	RAG cross-tenant canary and injection tests	Pipeline results	Zero canary leaks	AI Lead
7	Agent tool matrix and approval tests	Test evidence	Unapproved and prohibited calls denied 100%	AI Lead
8	Logging coverage and alert fire drill (D02, D03, D06, D07, D13)	SIEM records	Every alert fired with timestamp; heartbeat healthy	Security
9	Incident plan and one tabletop completed	Plan and tabletop record	Roles named; notification decision path tested	Security + Compliance
10	LLM provider DPA, residency and retention decision; PHI-to-LLM flag state	Legal sign-off; egress logs	Approved in writing, or flag remains blocking	DPO
11	Dependency, image and secret scan results; SBOM	CI reports	No open critical/high in reachable code; no live secrets	DevOps
12	Risk register with named owners and signed exceptions	Register	Every open item owned, dated and approved	CTO

19.4 Recommendation

Conditional go. Proceed with the customer demonstration on an isolated tenant with synthetic data. Approve production use with real patient data only when evidence items 1-10 are green, and re-review at Day 30. This lets the business keep its timeline without accepting risks we can already name.

Part 20. Ten Findings the Day Before a Customer Demonstration

Bottom line. Fix what exposes patient data or credentials on the demo path; mitigate what cannot safely be fixed in time; accept what is low and cheap to explain; defer the rest with dates. The most powerful move is to run the demo on an **isolated tenant with synthetic data**, which shrinks the impact of several findings at once.

> **Assumption A7:** the brief did not include the ten findings. The list below is illustrative, drawn from the Part 1 themes and typical test results. The decision rules apply unchanged to real inputs.

20.1 Decision rules

Decision	Use when	Condition
FIX	Exposes patient data or credentials, or is reachable on the demo path	Fix fits in about 4 hours with a rollback; otherwise treat as MITIGATE
MITIGATE	Real risk but a safe fix will not fit	A compensating control (flag, restriction, isolation) is in place and tested; fix scheduled with an owner
ACCEPT	Low risk; cost or disruption exceeds benefit	Named owner signs; expiry date; no PHI or credential involved
DEFER	Not on the demo path and no PHI or credential exposure	Tracked with a date inside the Part 8 SLA

20.2 Ranking (highest risk to the demo first)

Rank	Finding (illustrative)	Decision	Action before the demo	Time	Owner	Residual / follow-up
1	Production database firewall allows developer IP ranges	**FIX**	Remove the rule; confirm private only; scan from outside	30 min	DevOps	None; JIT path for emergencies
2	Admin panel reachable from the internet, SSO without MFA	**FIX**	Restrict to VPN/ZTNA egress; enforce MFA; block path at gateway	1-2 h	DevOps + IT	Full ZTNA within 14 days
3	Live LLM API key found in Git history	**FIX**	Rotate; move to secrets manager; scan history (Part 4)	1-2 h	DevOps	History rewrite as hygiene later
4	/documents/{id} lacks object-level authorization	**FIX**	Add tenant and entitlement check plus regression test. If not done by T-12h: MITIGATE with synthetic-only demo tenant and a flag	2-4 h	Backend	Full authZ-aware RAG in 30 days
5	Prompt and response bodies containing patient identifiers are logged	**FIX**	Turn off body logging; restrict the log store; schedule purge after Legal confirms holds	1 h	Backend + DPO	Purge and redaction at source
6	Agent can call update_customer and send_email without approval	**MITIGATE**	Disable both in the demo tenant, or require human approval; keep read-only tools	2 h	AI Lead	Tool gateway and approvals in 2 weeks
7	No rate limiting on /assistant/message	**MITIGATE**	Gateway rate limit and per-tenant token cap	2-3 h	Backend	Proper quotas next sprint
8	Internal worker container runs as root	**DEFER**	Ticket with 30-day date; not on demo path, no PHI exposure	n/a	DevOps	Non-root and admission policy by Day 30
9	Medium dependency CVE; vulnerable function not reachable	**DEFER**	Patch in next release inside the 90-day SLA	n/a	Backend	Tracked in the register
10	Server version disclosed in response headers	**ACCEPT**	Owner signs; expires in 30 days; remove header when convenient	n/a	Backend Lead	Low; no exposure of data

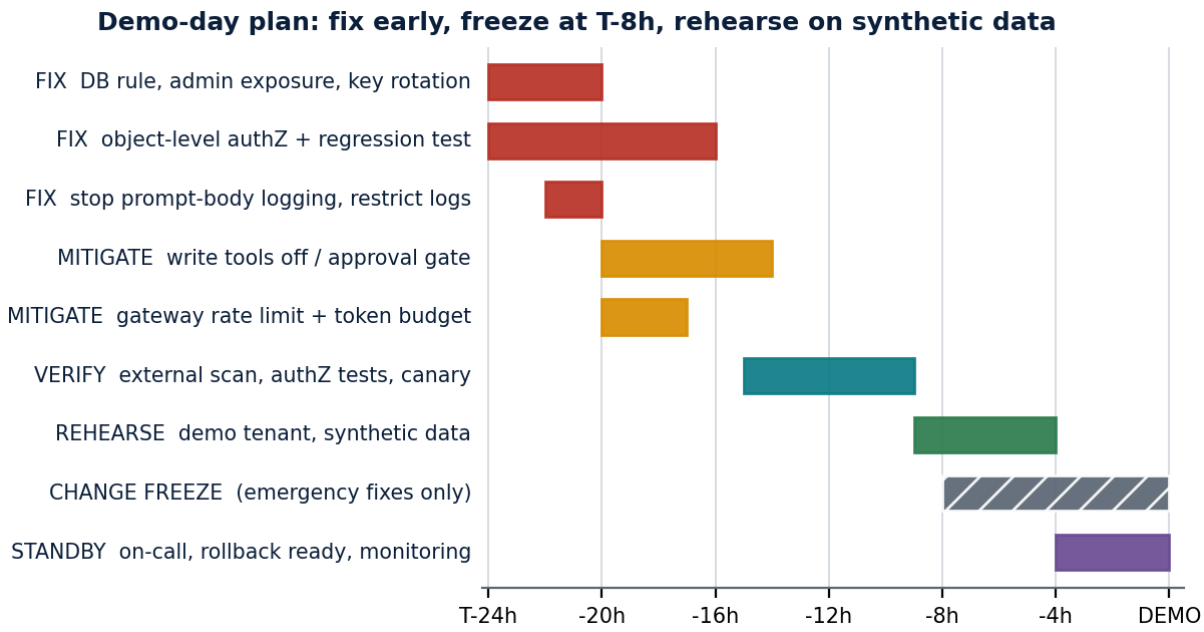


Figure 20.1. Timeline: fixes first, verification, change freeze at T-8h, rehearsal on synthetic data, standby.

20.3 Go / no-go and fallbacks

- **Go if**: findings 1-3 and 5 are verified fixed; finding 4 is fixed or the demo uses the synthetic-only tenant; findings 6-7 have tested mitigations; an on-call and rollback plan exist.
- **If a fix fails**: roll back, switch the affected feature off by flag, and demonstrate without it. A smaller demo beats an insecure one.
- **Change freeze at T-8h**: only emergency security fixes, each with the CTO's approval, to avoid breaking the demo with late changes.
- **Tell the customer?** Not required to disclose internal hardening, but do not claim controls we have not evidenced. Answer questions with the evidence list in Part 19.

Bonus. A Respectful Challenge (400 words maximum)

Instruction I would challenge. The senior leader's position in Part 19: "*We use HTTPS, a major cloud provider and a managed database, so we are secure enough.*" The implied instruction is to go ahead with the customer demonstration and launch with patient data as-is, with no further security work.

What I would accept. The goal and the foundations. Speed matters, the customer date is real, and HTTPS, a major cloud and a managed database are good building blocks that remove real work (patching, availability, encryption in transit). I would not ask to delay the whole programme.

What I would change. "Secure enough" is a claim, not evidence. Those choices protect the connection and the hardware, not our logins, our configuration or our AI features, which is where our findings sit. I would change the instruction to a **conditional go**: run the demonstration on an isolated tenant with synthetic data, and allow real patient data into production only when a short list of evidence is met (MFA on privileged accounts, no shared admin, database and admin interface unreachable from the internet and developer addresses, a tested restore, cross-tenant and agent tests passing, alerts drilled, LLM provider terms approved).

Why. A patient-data incident in front of a healthcare customer costs far more than a week of hardening: the contract, regulator attention and reputation. Several findings (no MFA, shared admin, patient data in prompt logs) are cheap to fix and expensive to explain after an incident. Legal exposure under the Personal Data Protection Law needs Legal to confirm. Requires legal/compliance validation.

How I would preserve speed. The one-day plan is ten configuration changes or feature flags, run in parallel by the existing leads, each with a proof. The demonstration does not wait for the deeper work (SIEM tuning, penetration test, compliance mapping), which follows a 30/60/90 roadmap. Synthetic data removes approval delays and keeps the date.

How I would raise it. Privately with the executive first, in business terms with the evidence list attached, offering the date-saving path rather than a refusal.

Word count: 340.

AI and Tools Disclosure

The assessment allows AI tools if disclosed. This section is candid about what they did and what still needs a human.

Item	Disclosure
AI tool used	Claude (Anthropic). It drafted the structure, prose, tables, diagram code and the detection-engineering code, and cross-checked coverage against the brief.
Other tools	Python 3 (standard library, unittest), matplotlib (diagrams and charts), ReportLab (PDF).
What AI accelerated	First drafts, code scaffolding, formatting, and consistency checks across 20 parts.
Verified by execution	50 unit tests, the 12-mutant check and the evaluation run were executed; their outputs are in the evidence folder. The mutation check found one test gap, which was fixed.
Not independently verified	Regulatory references (NCA ECC subdomain numbering, PDPL topics and deadlines, ISO, NIST and CIS mappings), product and tool names, risk scores and thresholds, the Dockerfile build, YAML and shell snippets other than those in the evidence folder.
Human judgement still required	Confirming that the scores, priorities and thresholds are ones the candidate will defend; validating legal and regulatory statements against source documents; adapting to Apex's real environment.

Candidate declaration (to complete before submission)

☐	I have read and understood every section and can explain it without notes.
☐	I have reviewed and, where I disagree, changed the judgements (scores, priorities, thresholds, decisions) that I will defend.
☐	I have checked regulatory and framework references against the source documents or asked Compliance/Legal.
☐	I have reproduced the evidence by running the commands in Part 18.
	Name: _____ Date: _____

Appendix A. Live-Defence Preparation: A Surprise Incident

The live session may add an unseen scenario. This page is the method I would use, and one worked example: **an AI agent's credential is suspected compromised**.

A.1 Five questions for any surprise scenario

- **What exactly happened, and what is the asset?** Facts first; label everything else as an assumption.
- **What is the blast radius?** What can this identity or system reach? (Design goal: small by construction.)
- **What is the least disruptive reversible action I can take in the next hour?** Usually revoke, block, flag-off, isolate, snapshot.
- **Who must know, and by when?** Owner, CTO, DPO/Legal; customer and regulator clocks.
- **How do we know it worked and how do we stop it recurring?** A check that proves containment, then a control and a detection.

A.2 Worked example: agent credential compromised

Window	Actions
0-15 min	Declare SEV-1 if PHI tools are in scope. Flip the kill switch for the agent (feature flag). Revoke the agent's tokens at the token-exchange service; if it holds a standing credential (today's state), rotate it now. Preserve tool-gateway and egress logs.
15-60 min	Query the tool gateway for every call by that agent identity in the exposure window: tool, parameters hash, approval ID, on-behalf-of user, decision. Check alerts D09-D11. Identify affected tenants, records and any writes or emails sent. Check whether the token reached systems outside the gateway.
1-4 h	Determine origin: prompt injection, leaked secret, compromised runtime, or insider. Rotate signing keys if token minting could be affected. Reverse unauthorised changes from audit logs. Legal and DPO decide notifications.
4-24 h	Restore agent with reduced tools (read-only) under enhanced monitoring; add the missing detection or control; regression-test the abuse path; blameless review.

Design argument: with per-tool, on-behalf-of tokens that live 5-15 minutes, a stolen agent credential expires quickly and can only do what one user could do with one tool. With today's broad agent credentials (finding #15) the same event is a major incident. That contrast is the reason finding #15 is in the top five.

Appendix B. Likely Questions in the Live Defence and Short Answers

#	Question	Short answer
1	**Why identity before the database or the AI issues?**	Control-plane compromise voids every other fix, and MFA plus retiring the shared admin takes hours. The database rule and admin exposure follow the same day.
2	**How defensible are your scores?**	They are judgement on stated anchors (Part 1). Ordering inside the Critical band comes from attack-path and effort, not the number. I would re-rank if I learned, for example, that the admin interface is already behind a VPN.
3	**Demo is tomorrow and MFA will slow developers. What now?**	Enforce MFA for admins today (hours), staff in a day. Keep break-glass. The cost is minutes per login; the alternative is account takeover with no attribution.
4	**Would your detections have caught the 25,000-record incident, and how fast?**	In synthetic replay D06 fired at about 130 s with 5,000 records read, and D07 correlated the egress. Detection limits loss; a gateway record budget bounds it.
5	**How do you know the rules work on real data?**	I do not. The data is synthetic and self-authored. Plan: alert-only for 2-4 weeks, tune per actor, then a purple-team replay.
6	**Prompt injection cannot be prevented. So what is the point of your controls?**	Assume the model will sometimes be steered. Authorization, tenant isolation, approvals and egress control sit outside the model, so a steered model cannot reach what it may not touch.
7	**Why is search_customer Restricted rather than Autonomous?**	It reads personal data and is the natural enumeration tool. It is allowed only with an exact identifier, small results, user's own permissions and rate limits.
8	**Why not purge the prompt logs today?**	They may be under legal hold or needed to scope exposure, and purging is irreversible. Restrict access now, stop new PHI, and purge after Legal confirms.
9	**Rotate or revoke first?**	Any sign of misuse or a public repo: revoke now. Otherwise replace, deploy and revoke within an hour. Never wait for the investigation.
10	**You get four hours. What do you do?**	Database rule, admin exposure, MFA for admins, disable agent write tools. Those cut the shortest paths to patient data.
11	**What are you least sure about?**	Regulatory specifics (article numbers, deadlines), the numeric thresholds, and cloud-specific implementation. I flagged each as [Validate] or [Design].
12	**A developer says JIT access is too slow. Response?**	Measure it. Provide the safer alternatives (synthetic data, masked views, admin tooling) so JIT is rare, and keep a fast on-call approval for the exceptions.
13	**Should Apex use an external LLM at all?**	Yes, conditionally: through an egress gateway with tokenization, approved terms and residency, and a kill switch. For the most sensitive flows use an approved in-Kingdom model.
14	**How would you show security is improving?**	Percent privileged users on MFA, time to remediate by tier, open exceptions, cross-tenant canary hits (zero), alert fire-drill pass rate, restore-test success and time.
15	**What did AI do here and how did you verify?**	It drafted and coded; I list what was executed (tests, evaluation) versus not verified (regulatory references) in the disclosure.

Appendix C. Evidence Index and Files

File or folder	Purpose
Apex_Cybersecurity_Assessment_Solution_Pack.pdf	This document
risk_register.csv	Editable copy of the 19-finding register (Part 1)
docker/Dockerfile.insecure, Dockerfile.hardened, .dockerignore, pod-securitycontext.yaml	Part 6 artefacts. Not built in my environment [Design]
detection_engineering/engine.py, config.json	16 detection rules and thresholds (Parts 10, 18)
detection_engineering/generate_logs.py, data/	Synthetic data generator and generated logs (seed 42)
detection_engineering/test_rules.py, mutation_check.py	50 tests and the 12-mutant check
detection_engineering/run_eval.py, results/	Evaluation outputs: alerts (json, csv), evaluation.json, threshold_sensitivity.csv, s4_timeline.json, test_output.txt, mutation_check.json
detection_engineering/README.md	Purpose, limitations and reproduction steps

All executed evidence is on synthetic data. Nothing was run against Apex systems or customer data.